DESIGN AND IMPLEMENTATION OF A WEB-BASED HOTEL MANAGEMENT AND BOOKING SYSTEM: A CASE STUDY OF DEUTCHEZZ HOTEL ISPAHAN

BY

[TO BE PROVIDED: student full name]

[TO BE PROVIDED: matriculation number]

A DISSERTATION SUBMITTED TO THE COMPUTER SOFTWARE ENGINEERING TECHNOLOGY DEPARTMENT, ZIBEH INSTITUTE OF TECHNOLOGY, JOS, IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE NATIONAL INNOVATION DIPLOMA (NID)

[TO BE PROVIDED: month and year of submission]

DECLARATION

I hereby declare that the work presented in this report, whose title is "Design and Implementation of a Web-Based Hotel Management and Booking System: A Case Study of Deutchezz Hotel Ispahan", was produced by me alone and is my own original work and research. It has not been presented for the award of any other diploma or degree in this or any other institution. All quotations are clearly indicated and the sources of information used have been acknowledged by means of reference.

[TO BE PROVIDED: student full name]

[TO BE PROVIDED: matriculation number]

[TO BE PROVIDED: signature and date]

CERTIFICATION

This document certifies that the candidate named below carried out the work described under the title "Design and Implementation of a Web-Based Hotel Management and Booking System: A Case Study of Deutchezz Hotel Ispahan", in partial fulfilment of the requirements for the award of the National Innovation Diploma in Computer Software Engineering Technology at Zibeh Institute of Technology, Jos, Plateau State, Nigeria.

[TO BE PROVIDED: project supervisor]

Project Supervisor - Date

[TO BE PROVIDED: head of department]

Head of Department - Date

DEDICATION

[TO BE PROVIDED: personal dedication text]

ACKNOWLEDGEMENT

[TO BE PROVIDED: acknowledgement text]

ABSTRACT

This work produced a web-based hotel management and booking system for Deutchezz Hotel Ispahan, a small hotel in Jos, Plateau State, Nigeria, that had managed its rooms, reservations, guests and payments by hand. The system centralises those records, lets a guest discover available rooms for a chosen date range and create a reservation online, and separates the public booking site from the receptionist and administrator consoles through server-side role-based access control. It was built as a four-layer ASP.NET Core application with a React and TypeScript frontend and a PostgreSQL database, in which availability, per-night prices, the reservation lifecycle and the late-check-out surcharge are computed on the server, and booking creation is protected by an idempotency guard and row-level locking so that two overlapping reservations cannot be confirmed for the same room. The domain, architecture and frontend test suites were executed successfully, while the integration suite that exercises the booking, availability, reception and administrative paths against a database could not be run in the environment used for this report. Because no formal acceptance session or online payment gateway was available, payments are recorded at the front desk and these items are stated as future work.

TABLE OF CONTENT

DECLARATION 2

CERTIFICATION 3

DEDICATION 4

ACKNOWLEDGEMENT 5

ABSTRACT 6

TABLE OF CONTENT 7

LIST OF TABLES 12

LIST OF FIGURES 13

1.0 Background 14

1.1 Statement of the Problem 15

1.2 Aim and Objectives of the Study 16

1.2.1 Aim of the Study 16

1.2.2 Specific Objectives of the Study 16

1.3 Research Questions 17

1.4 Scope and Limitations 17

1.4.1 Scope of the Study 17

1.4.2 Limitations of the Study 18

1.5 Significance of the Study 18

1.5.1 Significance to the Hotel 18

1.5.2 Significance to Guests, Developers and Researchers 19

1.5.3 Justification for the Choice of an Owned System 19

1.6 Definition of Terms 20

2.0 Introduction 22

2.1 Theoretical Framework 22

2.1.1 Hotel Management and Reservation Systems 22

2.1.2 Web Application Architecture and the REST Style 23

2.1.3 Relational Databases and Concurrency Control 24
2.1.4 Authentication, Authorisation and Application Security 25
2.1.5 API Contract Conventions and Reliability 26
2.1.6 Agile Software Development 26
2.2 Related Works 27
2.2.1 Mwandila (2026): Online Hotel Reservation System for Lundazi Castle Hotel 27
2.2.2 Williams and Micheal (2018): Reservation Management for Grand Ville Hotels 28
2.2.3 Commercial Platform: Booking.com 28
2.2.4 Commercial Platform: Cloudbeds 29
2.2.5 Open-Source Solution: QloApps 29
2.3 Comparative Discussion and Research Gap 30
2.3.1 Implications of the Gaps for the Proposed System 32
2.3.2 Justification for the Proposed System over the Reviewed Solutions 32
2.4 Summary 33
3.0 Introduction 35
3.1 Design 35
3.1.1 Research Methodology 35
3.1.1.1 Agile Development Methodology 36
3.1.1.2 Method of Data Collection 36
3.1.1.3 Development and Documentation Tools 37
3.1.2 Analysis of the Existing System 38
3.1.2.1 The Existing System 38
3.1.2.2 Weaknesses of the Existing System 38
3.1.3 The Proposed System 39
3.1.4 Requirements Specification 41
3.1.4.1 Functional Requirements 41
3.1.4.2 Non-Functional Requirements 42

3.1.5 System Architecture 43
3.1.5.1 Architecture Overview 43
3.1.5.2 Backend Layering 44
3.1.5.3 Request Lifecycle and Cross-Cutting Concerns 45
3.1.6 Use Case Modelling 46
3.1.6.1 Actors 46
3.1.6.2 Use Case Descriptions 47
3.1.6.3 Role-Based Access Control 49
3.1.7 Database Design 50
3.1.7.1 Entity-Relationship Model 50
3.1.7.2 Data Dictionary 51
3.1.7.3 Migrations and Schema Evolution 53
3.1.8 Data Flow and Process Modelling 54
3.1.9 System Modules 57
3.2 Technology Stack 58
3.3 Implementation 60
3.3.1 Booking Lifecycle and State Transitions 60
3.3.2 Availability and Concurrency 61
3.3.3 Idempotent Booking Creation 62
3.3.4 Duration-Based Pricing 63
3.3.5 Validation and Error Handling 64
3.3.6 Security Implementation 65
3.4 Testing Methodology 66
3.4.1 Unit Testing 66
3.4.2 Integration Testing 66
3.4.3 Architecture Testing 67
3.4.4 Frontend Testing 67

3.4.5 Functional Testing 67
3.5 Summary 68
4.0 Introduction 69
4.1 System Implementation Overview 69
4.2 User Interfaces and Features 71
4.2.1 Room Discovery 71
4.2.2 Availability Search and Pricing Quote 72
4.2.3 Reservation and Booking Confirmation 73
4.2.4 Registration and Email Verification 74
4.2.5 Customer Account 75
4.2.6 Profile and Notification Handling 76
4.2.7 Receptionist Console 76
4.2.8 Walk-in Booking 77
4.2.9 Check-in, Check-out and Overstay Charge 78
4.2.9.1 Room Assignment, Reassignment and Room-Type Change 78
4.2.10 Payment Recording 79
4.2.11 Administrator Console 80
4.2.12 Notifications and Email 81
4.3 API Implementation 81
4.4 Functional Evaluation 82
4.5 Reliability and Security Evaluation 84
4.6 Testing Results 86
4.7 Discussion: Alignment with Objectives and Research Questions 88
4.8 Comparison with Related Systems 89
4.9 User Acceptance Testing Status 90
4.10 Summary 91
5.0 Introduction 92

5.1 Summary of the Study 92

5.2 Achievement of the Objectives 93

5.3 Answers to the Research Questions 95

5.4 Findings 96

5.5 Conclusion 98

5.6 Recommendations 99

5.7 Suggestions for Further Work 100

5.8 Closing Remarks 101

APPENDIX A: API ENDPOINT CATALOGUE 105

APPENDIX B: CORE BUSINESS RULES REGISTER 108

LIST OF TABLES

Table 2.1: Comparison of the reviewed solutions and the proposed system 31

Table 3.1: Actors and their principal use cases 47

Table 3.2: Summary of the database entities 52

Table 3.3: Enumerations used by the system 53

Table 3.4: System modules 58

Table 3.5: Technology stack 59

Table 3.6: The duration-based pricing rule 64

Table 4.1: API controllers and the number of endpoints in each 71

Table 4.2: Automated test suites and the outcome of the executed runs 86

Table 4.3: Verification status of the principal scenarios 88

Table A.1: Complete API endpoint catalogue 107

Table B.1: Core business rules register 110

LIST OF FIGURES

Figure 3.1: High-level client-server architecture 44

Figure 3.2: Backend layer structure and allowed dependencies 45

Figure 3.3: Use case diagram for the customer, receptionist and administrator actors 49

Figure 3.4: Entity-relationship diagram 51

Figure 3.5: Activity diagram for the online booking process 56

Figure 3.6: Sequence diagram for booking creation 57

Figure 4.1: Room-type listing on the public site 72

Figure 4.2: Room-type detail with images, features and price 72

Figure 4.3: Availability search form and results with prices 73

Figure 4.4: Booking form with the computed price 74

Figure 4.5: Booking confirmation showing the reference and stay details 74

Figure 4.6: Registration form 75

Figure 4.7: Email verification code entry 75

Figure 4.8: Customer dashboard listing bookings 75

Figure 4.9: Customer booking detail and notifications 76

Figure 4.10: Reception console booking list and search 77

Figure 4.11: Room assignment and check-in 77

Figure 4.12: Check-out with the overstay charge and payment recording 77

Figure 4.13: Administrator dashboard and reports 80

Figure 4.14: Room and room-type management, including surcharge tiers 80

Figure 4.15: Staff management and activity log 81

CHAPTER ONE

INTRODUCTION

1.0 Background

Digital information systems are now central to the operation of modern hospitality businesses. A hotel must keep an accurate record of its rooms, of the reservations made against those rooms and of the guests who hold them, and it must be able to confirm a reservation without risk of selling the same room twice. Keeping such records by hand or on paper is slow, difficult to audit and easily made wrong, and it makes a single booking hard to follow from the first enquiry through to check-out (Williams & Micheal, 2018). Research into online hotel reservation systems reports that a web-based platform improves the efficiency of hotel operations while giving guests a more convenient and transparent booking experience (Mwandila, 2026). These findings reflect the situation at Deutchezz Hotel Ispahan, which manages a substantial volume of room and guest information and had no single system through which that information could be kept and used. The proposed system addresses that situation by providing an integrated web-based platform for room and reservation management at the hotel.

Deutchezz Hotel Ispahan is a hotel located along Ahmadu Bello Way in Jos, Plateau State, Nigeria. Its operation depends on a small front desk that receives enquiries, records reservations, assigns physical rooms, checks guests in and out and records payments, and on an administrator who maintains the room inventory and oversees the operation. The system described in this report replaces the separate, manual records used for these activities with a single application in which the room inventory is the shared source of truth, availability is computed from the reservations that exist, and every front-desk operation is recorded against the reservation it affects.

The system was designed around a small but complete set of hospitality concepts. A hotel offers room types, each of which carries a nightly price, a description and a set of features; each room type has a number of physical rooms at particular positions in the building. A guest makes a reservation for one or more room types over a date range, and the physical rooms are assigned later by the front desk. This separation between what a guest reserves and which physical room is used is a deliberate design decision, because the two concerns are handled by different people at different times. The background to the study therefore draws together the management of hotel records, the calculation of availability and price, and the operational workflow of the front desk, and it is these three concerns that the implemented system unifies.

1.1 Statement of the Problem

Deutchezz Hotel Ispahan manages room, reservation and guest records whose volume and pace make manual handling unreliable. When reservations are recorded by hand, it is difficult to answer a simple question such as how many rooms of a given type remain free for a particular night, and it is easy to record two reservations for the same physical room. The absence of a single availability check means that overbooking can only be detected after the fact, at the point where two guests are being shown to the same room. Reconciling the money taken at the desk against the reservations it belongs to is also difficult when payments are not tied to a booking record, and it is not possible to reconstruct what happened to a reservation when the events are not logged.

Guests face a related difficulty. Without an online channel, a guest cannot discover which rooms are available for the dates they intend to travel, cannot see the price of those rooms before arriving, and cannot hold a reservation without telephoning or visiting the hotel. A guest who books online expects to receive a confirmation that can be produced at check-in

and, where the policy of the hotel allows it, a clear route to cancel the reservation. The problem that this project addresses is therefore the absence of an integrated system that keeps the hotel's room and reservation records accurately, computes availability and price consistently, and gives guests a dependable online booking path.

1.2 Aim and Objectives of the Study

1.2.1 Aim of the Study

This work set out to build a web-based hotel management and booking system for Deutchezz Hotel Ispahan that centralises room, reservation and guest records, computes availability and price on the server, and supports both self-service booking by guests and the operational workflow of the hotel's front desk.

1.2.2 Specific Objectives of the Study

The study pursued the objectives below:

1. To provide a facility for creating, updating and deactivating the hotel's room types and physical rooms.

2. To enable a guest to check which rooms are free for a chosen date range and to reserve them online.

3. To keep reservations, guests and payments in an accurate record and to stop any physical room from being sold twice.

4. To give the front desk a console for allocating rooms, checking guests in and out, taking payments and entering walk-in reservations.

5. To give management a console for maintaining rooms, room types, staff and settings and for obtaining reports and an audit trail.

6. To assess the delivered system against these objectives and against the reliability and security requirements of the problem.

1.3 Research Questions

The study sought answers to the questions below:

Which architecture and design are appropriate for a hotel management and booking system of this kind?

How should available rooms be determined for a requested date range so that a physical room cannot be booked twice?

How should reservations and guests be recorded so that the records are accurate and can be traced?

How should front-desk operations such as room assignment, check-in, check-out and payment recording be supported?

How should room inventory and the hotel's administrative records be managed through the system?

1.4 Scope and Limitations

1.4.1 Scope of the Study

The study covers the design and implementation of a web-based hotel management and booking system for a single property, Deutchezz Hotel Ispahan. The system manages room types and their nightly pricing, physical rooms and their operational status, date-based availability, online and walk-in reservations, guest records, room assignment, check-in and check-out, payment recording, and the administrative management of rooms, room types, staff, settings, reports and audit logs. It supports three roles, namely the customer, the

receptionist and the administrator, and it enforces the permissions attached to each role on the server. Pricing is calculated by the backend at a nightly rate for each room type, with an additional duration-based surcharge applied only when a stay runs beyond its booked nights. The system does not include an online payment gateway, a housekeeping module, a loyalty programme, dynamic or seasonal pricing, or multi-property management; these are described as future work in Chapter Five.

1.4.2 Limitations of the Study

Several limitations should be stated. The integration test suite of the system requires a running PostgreSQL instance, and the environment used for this report did not complete an integration test run, so the evaluation in Chapter Four states the evidence on which each result rests and does not cite a test count that was not produced. No acceptance test involving staff of the hotel had been carried out when this report was written, so the report describes the behaviour that was built rather than measured user satisfaction. No response-time or load-testing measurements were taken, and the report therefore makes no performance claims. Payment is taken at the hotel and recorded by the front desk, so the study does not evaluate an online payment flow. Finally, the findings are limited to the single case study and are not presented as general to all hotels.

1.5 Significance of the Study

1.5.1 Significance to the Hotel

The system is significant to the management of Deutchezz Hotel Ispahan because it replaces fragmented manual records with a single, shared and auditable source of truth for rooms, reservations and guests. It gives the front desk a reliable availability check, prevents the same physical room from being sold twice, ties each payment to the reservation it belongs to, and records the important operational and administrative actions for later review. Taken together,

these changes reduce the risk of the operational errors that the statement of the problem identifies.

1.5.2 Significance to Guests, Developers and Researchers

The system is significant to guests because it provides an online channel through which rooms can be discovered and reserved, and because it produces a booking reference through which a reservation can be looked up or cancelled. It is significant to developers and researchers because it demonstrates how a compact hospitality domain can be implemented with clear separation of concerns, backend-authoritative pricing and availability, and explicit reliability and security controls, and because it documents the design decisions and their trade-offs for reuse in comparable systems.

1.5.3 Justification for the Choice of an Owned System

A hotel of this kind has three practical alternatives to the system developed here: to rely on a commercial online travel agency such as Booking.com, to subscribe to a commercial property management system such as Cloudbeds, or to adopt an open-source platform such as QloApps. Each of the three was reviewed in Chapter Two, and each carries a cost or a limitation that does not suit this case study, as Section 2.3.2 sets out in detail. In brief, the online travel agency takes a commission on every booking it produces and holds the guest relationship with the platform rather than with the hotel, and it does not manage the hotel's internal operations of room assignment, check-in, check-out and payment recording, so the hotel would still require a management system of its own. The commercial property management system is priced as a recurring subscription, commonly per room per month, and much of its capability is surplus to a single small property. The open-source platform avoids a licence fee but is built on a different technology stack and is configured for a general booking website rather than for this hotel's own rules.

The decision was therefore taken to build an owned and tailored system rather than to adopt one of the reviewed alternatives. The delivered system keeps the hotel's data and its full room revenue, implements the hotel's own cancellation window, no-show handling and duration-based surcharge directly, and applies the reliability controls - a server-authoritative availability check, database row locks and idempotent booking creation - that stop the same room from being sold twice. Chapter Two acknowledges that the commercial platforms exceed the delivered system in breadth of features, vendor support and multi-property handling; the justification advanced here is specific to a single small property of the size of the case study.

1.6 Definition of Terms

Hotel Management System: A software application that supports the operation of a hotel by managing its rooms, reservations, guests, payments and front-desk activities.

Reservation: A record of a guest's intent to occupy one or more room types over a stated date range, held against the hotel's inventory.

Room Type: A category of room, such as Standard, Deluxe or Executive, that carries a nightly price, a description, a maximum occupancy and a set of features.

Room: A single physical unit of a room type, identified by a room number and a floor, and carrying an operational status such as Available, Occupied, Maintenance or Inactive.

Availability: The condition in which at least one active room of a requested type is free of conflicting reservations for the dates requested.

Overbooking: The error of assigning or confirming more rooms of a type than the hotel can physically provide for the same period.

Walk-in Booking: A reservation created by the front desk for a guest who arrives without a prior booking, and for whom no account or email address is recorded.

Booking Reference: A unique, human-readable identifier, in the form DHI-YYYY-XXXXXX, by which a reservation can be quoted and looked up.

Check-in: The front-desk operation that marks a confirmed reservation as being in house and marks its assigned physical rooms as occupied.

Check-out: The front-desk operation that marks a stay as complete and releases its assigned physical rooms back to availability.

Role-Based Access Control: The enforcement of permissions according to the role held by the authenticated user, in this system the customer, the receptionist or the administrator.

Idempotency: The property by which repeating the same request with the same key produces the same result without creating a duplicate record.

Pessimistic Concurrency Control: The use of database row locks to prevent two transactions from changing the same rows at the same time.

JSON Web Token: A signed, time-limited token that carries a user's identity and role and is presented on authenticated requests.

Property Management System: A software system used by hotels to manage reservations, rooms, guests and front-desk operations (Cloudbeds, n.d.).

Online Travel Agency: A commercial platform, such as Booking.com, that markets accommodation from many providers to consumers and takes a commission on bookings (Booking.com, n.d.).

CHAPTER TWO

LITERATURE REVIEW

2.0 Introduction

This chapter reviews the knowledge on which the proposed web-based hotel management and booking system for Deutchezz Hotel Ispahan is built. Section 2.1 develops the theoretical framework, covering hotel management and reservation systems, web application architecture and the REST style, relational databases and concurrency control, authentication and application security, API contract conventions, and agile software development. Section 2.2 reviews related works, including two academic hotel reservation systems, a commercial online travel agency, a commercial hotel property management system and an open-source hotel platform, and it ends by comparing them. Section 2.3 draws the comparison together and states the research gap that this project fills, and Section 2.4 summarises the chapter. The sources were drawn from peer-reviewed journals, standard texts and the official documentation of the products concerned, and they were chosen for their bearing on hotel operations, system architecture, data integrity and application security.

2.1 Theoretical Framework

2.1.1 Hotel Management and Reservation Systems

In this study, a hotel management system means software that keeps a hotel's rooms, reservations, guests, payments and front-desk activities together in one place. Williams and Micheal (2018) note that the management and booking of hotel rooms is a tedious and complicated task when it is done manually, because keeping track of a large number of customers and their details requires inordinate physical storage and repeated movement to and from filing cabinets. They argue that a database-driven reservation system solves the

problem by storing, retrieving and relating this information efficiently, and they develop their system using an object-oriented approach so that the resulting software is maintainable, reliable and scalable. Mwandila (2026) reaches a similar conclusion for an online hotel reservation system, reporting that an availability check, a streamlined booking process and a backend management interface together improve operational efficiency and the guest experience. These findings justify the design of the proposed system as a database-centred web application whose availability, pricing and state transitions are controlled by the backend rather than by the client.

The literature also clarifies the boundaries of the domain. Both Williams and Micheal (2018) and Mwandila (2026) distinguish the guest-facing booking process from the staff-facing management process, and both relate a reservation to a room through a database relationship rather than treating the room as the unit of reservation. This distinction is adopted in the proposed system, in which a guest reserves room types and quantities and the physical rooms are assigned later by the front desk. It is this separation that allows availability to be computed over dates while room assignment remains a separate operational concern.

2.1.2 Web Application Architecture and the REST Style

Modern hotel systems are commonly built as web applications composed of a client, a server and a data store. Fielding (2000) formalised the Representational State Transfer, or REST, architectural style, which describes the web as a set of resources identified by uniform resource identifiers, manipulated through a uniform interface of standard methods, and exchanged as self-descriptive representations. REST emphasises scalability, a general interface, independent deployment of components and the use of intermediary components, and it is the dominant style for application programming interfaces exposed over the web. The proposed system follows this style: a React single-page application runs in the browser

and communicates with a versioned RESTful API over HTTPS using JavaScript Object Notation.

Good software architecture is concerned not only with the style of the interfaces but with how responsibilities are separated inside a system. Gamma, Helm, Johnson and Vlissides (1994) catalogue recurring design solutions and argue that reusable designs emerge when responsibilities are named and separated rather than rediscovered in each project. Sommerville (2016) similarly presents architectural design as the activity of identifying the system's principal structural components, their relationships and the principles that govern their organisation. The proposed system applies these ideas by organising the backend into four logical layers, namely presentation, application, domain and infrastructure, and by enforcing the allowed dependency directions between them with automated architecture tests. This layering keeps HTTP concerns, business rules, the domain model and technical infrastructure separate from one another, so that each can be changed with limited impact on the others.

2.1.3 Relational Databases and Concurrency Control

A booking system is, at its core, a data system, and its correctness depends on the consistency of the data it stores. Kleppmann (2017) explains that concurrency and isolation are among the central difficulties of data-intensive applications, because several clients may attempt to change the same data at the same time and the database must prevent the resulting anomalies. The PostgreSQL Global Development Group (n.d.) documents the row-level locking facility of the database, in which a SELECT statement can lock the rows it retrieves so that other transactions cannot change them until the current transaction ends. Row-level locking taken in a consistent order is the mechanism that prevents the classic anomaly of a

booking system, in which two concurrent reservations each read the same remaining capacity and each conclude that the room is available.

The alternative to locking is optimistic concurrency, in which a record carries a version or concurrency token that is checked when the record is saved and an exception is raised if the record has changed since it was read (Microsoft, 2023). The proposed system uses both mechanisms, choosing between them according to the cost of the operation. Availability-affecting operations such as booking creation and room assignment take pessimistic row locks inside a transaction so that capacity cannot be exceeded, while edits to records such as a room type use an optimistic concurrency token so that a lost update is detected and reported rather than silently applied. This combination follows the guidance in the literature that the choice of concurrency-control strategy should match the operation's contention and the consequence of an anomaly (Kleppmann, 2017).

2.1.4 Authentication, Authorisation and Application Security

A hotel system holds personal and commercial data and exposes operations that change inventory, so its security controls must assume that requests may come from untrusted clients. The OWASP Foundation (2021) maintains the OWASP Top 10, a widely used consensus list of the most critical web application security risks, whose first two entries are broken access control and cryptographic failures. Broken access control covers the failure to enforce permissions so that a user acts outside the permissions of that user, and cryptographic failures cover the incorrect protection of data in transit and at rest. The proposed system responds to these risks by enforcing role-based authorisation on the server through named policies, by storing passwords with a modern password hash, by issuing access tokens that expire quickly and rotating a refresh token that is held only as a hash, and by not disclosing whether an account or an email address already exists. These controls are exercised in the

evaluation reported in Chapter Four, which records the responses returned to unauthenticated and wrongly authorised requests.

The literature on identity in web applications also draws attention to the verification of contact details and to the handling of credentials. The proposed system requires an email address to be verified before an account is created, hashes the verification code before storing it and compares it in constant time, and applies a lockout policy after repeated failed sign-in attempts. These practices follow the recommendations of the OWASP Foundation (2021) for identification and authentication, and they are described in more detail in Chapter Three.

2.1.5 API Contract Conventions and Reliability

An application programming interface is only as useful as the consistency of its contract. Nottingham and Wilde (2016) define the Problem Details format, a standard way for an HTTP API to carry machine-readable details of an error, so that clients do not need a bespoke error format for each service. The proposed system adopts this format for every error response and adds a machine-readable code, such as booking.insufficient_availability, to each one, so that a client can distinguish the reason for a failure without parsing a human-readable message. The literature also treats reliability as a design concern rather than an afterthought: Kleppmann (2017) argues that systems should be designed to behave predictably in the presence of faults, and the proposed system reflects this by making inventory-changing operations idempotent, by validating requests at the API boundary and by centralising the availability conflict check in a single service.

2.1.6 Agile Software Development

The methodology adopted for the study is agile. Beck and colleagues (2001) set out the values of agile software development, emphasising working software, close collaboration with the customer and responsiveness to change over comprehensive documentation and

rigid planning. Sommerville (2016) explains that agile methods deliver functionality in short, incremental cycles and that they suit projects whose requirements are expected to evolve. The proposed system was developed in iterative increments, each of which produced a working and tested part of the application, which allowed the requirements to be refined as the design became clearer. Section 3.1.1.1 describes how this methodology was applied.

2.2 Related Works

2.2.1 Mwandila (2026): Online Hotel Reservation System for Lundazi Castle Hotel

Mwandila (2026) presents the design and development of an online hotel reservation system for Lundazi Castle Hotel and its branches. The study begins by analysing existing reservation systems and derives functional and non-functional requirements from user feedback and market trends, and it then develops the system iteratively using an agile methodology. The delivered system provides a user-centric interface, real-time availability checks and a streamlined booking process, together with a backend management system for hotel staff, and the study reports that it improved the efficiency of hotel operations while providing guests with a satisfying booking experience. Its contribution is a clear demonstration of how an availability check and a staff-facing management interface can be combined in a single online system, and its use of agile development confirms that the approach suits a reservation-system project whose requirements evolve. Its limitation, from the perspective of a single hotel of the size considered here, is that the study does not describe the concurrency control that prevents two simultaneous reservations from consuming the same capacity, and it does not report a formal evaluation of the race conditions that arise when several guests book the same room type at the same moment.

2.2.2 Williams and Micheal (2018): Reservation Management for Grand Ville Hotels

Williams and Micheal (2018) design and implement a reservation management system for Grand Ville Hotels, using a database management system together with scripting and programming languages. The study adopts an object-oriented software development approach, including object-oriented analysis, design and programming, so that the resulting software is maintainable, reliable and scalable, and it describes the system through a three-tier architecture of presentation, middle and data tiers, with the database normalised to the third normal form. The system supports the recording of customer details, the making and updating of reservations and the generation of reports, and it treats the database as the authoritative store of reservation information. Its contribution is a clear treatment of the data model and the normalisation of a hotel reservation database, and its use of a layered architecture anticipates the layered design of the proposed system. Its limitation is that it does not address concurrent access to the availability data, and it does not provide an online guest channel with a live availability check; the reservations are managed primarily by staff.

2.2.3 Commercial Platform: Booking.com

Booking.com is a commercial online travel agency that markets accommodation from many independent providers to consumers. Its own company statements describe it as one of the world's leading online travel platforms, founded in 1996 and offering over thirty million reported listings across more than two hundred countries and territories, with hundreds of millions of verified guest reviews (Booking.com, n.d.). It connects travellers to hotels, homes and other places to stay, supports free cancellation at many properties, and provides round-the-clock customer support, while charging its accommodation partners a commission for bookings it generates. Its strengths are its scale, the depth of its inventory and the trust created by its review system. Its limitation, when judged against a single hotel that wishes to control its own records, is that it matches many suppliers of accommodation with many

consumers rather than providing a management tool for one property: it publishes availability but does not manage the hotel's internal assignments, check-ins, check-outs or payment records, and a hotel that relies on it alone still needs a separate system for its daily operation.

2.2.4 Commercial Platform: Cloudbeds

Cloudbeds is a commercial cloud-based hotel property management system that combines a property management system, a booking engine, a channel manager, payments and revenue management in one platform. Its own documentation describes it as serving independent hotels, hotel groups and alternative accommodation, with an online booking engine, a multi-property view, housekeeping, group management and reporting, and it reports integration with hundreds of distribution channels including large online travel agencies (Cloudbeds, n.d.). Its strengths are its breadth, its cloud deployment and its integration with the distribution ecosystem. Its drawbacks, when set against the needs of one small property such as the case study, are cost and complexity: it is a subscription product priced per room per month, and much of its capability, such as channel management, revenue management and central reservation, is surplus to the needs of a hotel that takes bookings directly and records payment at the desk. The proposed system deliberately implements the subset that a small hotel needs, and it does so as an owned application rather than a subscription.

2.2.5 Open-Source Solution: QloApps

QloApps is an open-source hotel management and reservation platform distributed under the Open Software License and developed as a community project on GitHub, where it presents itself as a platform that combines a property management system, a booking engine and a hotel website (QloApps, n.d.). It is built on PHP and MySQL, and it is intended to let a hotel launch its own booking website and manage reservations without licensing cost. Its strengths are that its source code is freely available and auditable, that it can be self-hosted, and that it

packages the booking and management functions that a hotel requires. Its limitations, for the case study, are that its technology stack differs from the one adopted here, that it is a large general platform whose configuration is not tailored to a single small property, and that it is oriented towards ownership and operation of a public booking site rather than towards the specific operational rules, such as the hotel's own cancellation window, no-show handling and duration-based surcharge, that the case study requires.

2.3 Comparative Discussion and Research Gap

The reviewed systems can be compared along three dimensions, namely the scope of the system, the approach taken to availability and concurrency, and whether the system is a single-property management tool or a multi-provider marketplace. Williams and Micheal (2018) contribute a well-normalised reservation database and a layered architecture but do not address concurrent access to availability or provide a live online guest channel. Mwandila (2026) contributes an online reservation system with an availability check and a staff interface but does not describe the concurrency control that prevents two simultaneous bookings from consuming the same capacity. Booking.com (n.d.) contributes scale and inventory but is a marketplace rather than a single hotel's management tool, and it does not manage the hotel's internal operation. Cloudbeds (n.d.) contributes a complete commercial platform but at a cost and breadth that exceed the needs of a small property. QloApps (n.d.) contributes an open and self-hostable platform but with a different stack and a general configuration. Table 2.1 summarises the comparison and adds the proposed system for reference.

System	Type	Cost model	Availability and concurrency	Data ownership	Suitability for a single small property
Williams & Micheal	Academic reservation	Not applicable	Normalised database;	Institution's own	Relevant data model, but no

(2018)	system	(research)	concurrent availability not addressed		online guest channel
Mwandila (2026)	Academic online reservation system	Not applicable (research)	Real-time availability check; concurrency control not described	Institution's own	Combines availability and a staff interface; concurrency gap
Booking.com	Commercial online travel agency (marketplace)	Commission on each booking	Publishes availability; internal operation not managed	Platform-owned	An extra marketing channel only; the hotel still needs an internal system
Cloudbeds	Commercial cloud property management system	Recurring subscription, per room per month	Vendor-managed	Vendor-hosted (lock-in)	Broader and more costly than the hotel needs
QloApps	Open-source platform	Free licence, plus hosting and maintenance	Platform-managed	Self-hosted	Different stack and a general configuration
Proposed system	Owned web application	One-off build; no commission and no subscription	Backend-authoritative, database row locks, idempotent booking	Hotel-owned	Tailored and right-sized (the target of this study)

Table 2.1: Comparison of the reviewed solutions and the proposed system

Taken together, the reviewed works leave a gap. None of them combines, in a single owned system tailored to one small property, a backend-authoritative availability and pricing computation, explicit concurrency control that prevents overbooking, a complete front-desk workflow of assignment, check-in, check-out and payment recording, and an online guest channel in one coherent application. It is this combination that the proposed system for Deutchezz Hotel Ispahan addresses.

2.3.1 Implications of the Gaps for the Proposed System

The gaps identified above shaped the design of the proposed system in three ways. First, because the literature reports that manual record-keeping is unreliable and that database-centred management improves accuracy (Williams & Micheal, 2018; Mwandila, 2026), the system treats the database as the authoritative record and computes availability, price and state transitions on the server rather than trusting the client. Second, because the reviewed academic systems do not fully describe their protection against concurrent bookings, the system treats concurrency as a first-class design concern and centralises the availability conflict check in a single service that uses database row locks inside a transaction. Third, because the commercial and open-source platforms are either broader than the case study requires or built on a different stack, the system implements the specific subset of hospitality functions that Deutchezz Hotel Ispahan needs, and it implements the hotel's own operational rules, such as its cancellation window, its no-show handling and its duration-based surcharge, directly in the application. Chapter Three presents the resulting design and implementation.

2.3.2 Justification for the Proposed System over the Reviewed Solutions

The comparison in Table 2.1 shows why a client with a single property of the size considered here should prefer the proposed system, and the case can be stated directly. First, on cost: the commercial travel agency takes a commission on every booking it creates, and the commercial property management system is priced as a recurring subscription, commonly per room per month, whereas the proposed system is built once and carries neither a commission nor a subscription, so every direct booking keeps the full room revenue with the hotel. Second, on ownership: the commercial platforms hold the guest relationship and the data, while the proposed system leaves the database, the guest records and the booking history with the hotel. Third, on fit: the reviewed platforms are either broader than a small property needs or configured for a general case, whereas the proposed system implements the

hotel's own cancellation window, no-show handling and duration-based surcharge directly, and applies the concurrency control that the two academic systems do not describe. Fourth, on reliability: because availability is settled on the server under database row locks and booking creation is idempotent, the system provides an overbooking guarantee that a client cannot inspect in a marketplace. Taken together, these points justify the decision to build an owned system rather than to adopt one of the reviewed alternatives. It is acknowledged, as stated in Section 2.3, that the commercial platforms exceed the proposed system in breadth of features, around-the-clock vendor support, proven scale and multi-property handling, and that a large hotel group would still be better served by a commercial property management system; the justification advanced here is specific to a single small property of the size of the case study.

2.4 Summary

This chapter reviewed the knowledge on which the system is built. Section 2.1 established the theoretical framework, drawing on the literature on hotel management and reservation systems, web application architecture and the REST style, relational databases and concurrency control, authentication and application security, API contract conventions and agile software development. Section 2.2 examined two academic reservation systems and three existing platforms, and Section 2.3 compared them in Table 2.1 and identified the gap that the proposed system fills, namely the absence of a single owned system for one small property that combines backend-authoritative availability and pricing, explicit concurrency control, a complete front-desk workflow and an online guest channel. Section 2.3 also explained how that gap shaped the design of the proposed system and justified the choice of an owned system over the reviewed commercial and open-source alternatives. Chapter Three presents the analysis and design of that system.

CHAPTER THREE

SYSTEM ANALYSIS AND DESIGN

3.0 Introduction

This chapter presents the analysis that informed the design of the system and the design itself. It begins with the design of the system, covering the methodology used to build the software, the way the requirements were gathered, the analysis of the manual system that the project replaces, the requirements for its replacement, and the architecture, actors, use cases, data model, process model and modules of the delivered system. It then sets out the technology stack chosen for the implementation and the principal implementation decisions, namely the lifecycle of a booking, the computation and protection of availability, the idempotent creation of bookings, the duration-based price, the validation and error conventions and the security controls. The chapter closes with the testing methodology and states the number and kind of automated tests that were written. The design is presented at a level of detail that allows the implementation described in Chapter Four to be understood and evaluated.

3.1 Design

3.1.1 Research Methodology

The study followed an applied design-and-implementation methodology in which the requirements were established from the operation of the case study and from the published literature, and the system was built in verifiable increments. Because the study produces a software artefact rather than a survey result, the problem was defined from the working practice of a small hotel, a design was proposed in response to it, and the design was realised and then examined against the problem it was meant to solve.

3.1.1.1 Agile Development Methodology

The study adopted an agile methodology. Beck et al. (2001) describe agile software development as a set of values that favour working software delivered frequently, close cooperation with the customer and the ability to respond to change over heavyweight documentation and fixed plans, and Sommerville (2016) explains that agile methods deliver a system in short increments so that requirements can be refined as understanding grows. Each increment of the project delivered a working and verifiable slice of the system, such as the room inventory, the availability search or the booking creation, and it was tested before the next increment began. A small set of automated tests was run to confirm that an increment had not broken an earlier one, so that the growing system remained in a working state.

This approach suited the case study because its requirements were refined through discussion rather than fixed in advance, and it meant that a usable part of the system existed early. The increments were ordered so that the data model and the availability engine were built first, because the later booking, reception and administrative features depend on them. Reception and administration were developed before the customer self-service features, because the operational workflows of the hotel had to be correct before a guest could be allowed to create a reservation through them. The ordering of the increments is recorded with the project's other design decisions so that the sequence can be understood after the fact.

3.1.1.2 Method of Data Collection

Secondary data were collected from the existing records and working practice of the case study and from the published literature and product documentation reviewed in Chapter Two. The purpose of the review was to establish how comparable systems handle availability, reservations and front-desk operations, and to confirm the technologies and controls that are accepted practice. Because the study is a design-and-implementation project rather than an

empirical survey, no questionnaire instrument was administered and no claim is made about user perceptions that was not observed.

Requirements were instead derived from three sources. The first was the operational workflow of a small hotel as seen at the front desk, from the first enquiry through the reservation, the assignment of a physical room, check-in, check-out and payment. The second was the published literature and product documentation reviewed in Chapter Two, which established how comparable systems handle availability, reservations and front-desk operations and confirmed the technologies and controls that are accepted practice. The third was the business rules recorded in the project's source of truth, which state the hotel's own policies, such as the cancellation window, the no-show handling and the duration-based overstay charge. No questionnaire was administered and no claim is made about a user's perception that was not directly observed, because the study is a design-and-implementation project rather than an empirical survey.

3.1.1.3 Development and Documentation Tools

The system was developed with the tools and technologies listed in Section 3.2. Version control was used throughout so that each increment was recorded and could be reviewed, and an automated documentation approach was used for this report so that the deliverable could be regenerated from a single source. The choice of a strongly typed backend and a typed frontend reduced a class of integration errors, and the automated tests described in Section 3.4 were run as part of the development cycle rather than only at the end. The design itself was recorded in a source-of-truth document that is versioned alongside the code, so that a change to a business rule is captured before the code that implements it changes.

3.1.2 Analysis of the Existing System

3.1.2.1 The Existing System

Before this project, the hotel recorded its rooms, reservations, guests and payments using manual methods, and its guest-facing channel was the telephone and the front desk. Room and guest details were written down and kept in the office, and the availability of a room type for a given date was worked out by whoever was on duty by reading through the existing records. There was no shared, authoritative record of which rooms were free on which nights, and the same information could be held in more than one place. Payments were recorded on paper, and it was not always possible to relate a particular payment to the reservation it belonged to.

The manual process followed a recognisable sequence. An enquiry was received at the desk or by telephone, the member of staff on duty consulted the written records to judge whether a room of the requested type was free, and, if it appeared to be free, a note of the reservation was made. The assignment of a physical room and the recording of payment happened later and were, in practice, separate from the reservation note. Because the three steps were recorded in different places, the correctness of the final record depended on the care of the individual member of staff rather than on any check performed by the records themselves.

3.1.2.2 Weaknesses of the Existing System

The weaknesses of the manual method follow from the absence of a single, validated record. Availability could not be checked with certainty, so two reservations could be made for the same physical room and the conflict was only discovered when a guest arrived. The information was slow to search and easy to duplicate, and it was vulnerable to loss and to illegible or incomplete entries (Williams & Micheal, 2018). Guests could not see which rooms were available or at what price without contacting the hotel, and they had no self-

service way to hold or cancel a reservation. There was no reliable record of who had changed a reservation or recorded a payment, so mistakes were difficult to trace, and reports for management had to be assembled by hand.

Three of these weaknesses are worth stating precisely, because the design of the proposed system answers them directly. The first is the absence of an availability check that is computed from the existing reservations, since a manual reading of the records cannot guarantee that a room has not already been promised. The second is the absence of a link between a payment and the reservation it settles, which makes reconciliation a matter of memory. The third is the absence of an audit trail, which makes it impossible to reconstruct who changed a record and when.

3.1.3 The Proposed System

The proposed system is a web application that keeps a single authoritative record of rooms, room types, reservations, guests, payments and operational events. It exposes a public website through which a guest can search for available rooms over a date range, obtain a price quote, create a reservation and look up or cancel it by its reference. It gives the receptionist a console for creating walk-in and phone bookings, assigning physical rooms, checking guests in and out, recording payments, handling late check-out and changing a room type. It gives the administrator an interface for managing room types, rooms, staff, settings, reports and the audit trail, and it computes availability, price and booking status on the server so that the client cannot override them.

The design separates three concerns that the manual process mixed together. The first is the reservation, which records what a guest has booked and for which dates, and which is the object against which availability is computed. The second is the assignment of a physical room, which happens later and is performed by the front desk. The third is the payment,

which is recorded against the reservation it settles. Keeping these three concerns distinct is what allows the system to answer the question that the manual method could not, namely how many rooms of a type remain free for a particular night, without confusing a reservation with the physical room that will later be used to serve it.

The backend is the authority for availability, price and state. A request that changes inventory is performed inside a database transaction that locks the rows it depends on, and a request that reads availability is answered from the same computation that a booking will use, so that what a guest is shown and what the guest is charged cannot disagree. This arrangement addresses the statement of the problem at its root: the system does not merely record what a member of staff has decided, it performs the check that the member of staff could not perform reliably.

The proposed system removes the weaknesses of the manual method in several concrete ways. It provides a single availability check that takes database locks before a room is committed, so a physical room cannot be sold twice (PostgreSQL Global Development Group, n.d.). It stores all records in one validated database, so information is not duplicated or lost and can be searched quickly. It gives guests an online channel that shows live availability and an accurate price before they commit, which the manual system could not offer. It records who performed each important operation through an activity log, so changes can be traced, and it assembles the reports that were previously prepared by hand. Finally, because it is a web application owned by the hotel, it avoids the subscription cost and the surplus breadth of the commercial products reviewed in Chapter Two (Cloudbeds, n.d.).

A further advantage is that the computation of price and availability is removed from the person of the member of staff and placed in the software. In the manual system the correctness of a reservation depended on the care of the individual, and a new member of

staff had to learn the rules by experience. In the proposed system the rules are expressed once, in code, and applied identically to every request, so that a correct result does not depend on which member of staff is on duty.

3.1.4 Requirements Specification

3.1.4.1 Functional Requirements

The functional requirements were grouped by the actor that they serve and by the system-wide behaviour that applies to all of them. They are stated as capabilities rather than as screens, because the same capability may be reached from more than one interface.

The customer requires the ability to browse the published room types and to see their nightly price, their features and their description; to search for availability over a requested check-in and check-out range and to see, for each room type, the number of rooms still available and the price for the stay; to obtain a price quote for a stay without making a booking; to create an account and verify their email address with a code; to create a reservation and receive a booking reference; to look up a reservation by its reference and email address; to cancel a reservation within the hotel's cancellation window; and to review their own bookings, profile and notifications once signed in.

The receptionist requires the ability to search and list reservations, including a filter for those with an outstanding balance and a free-text search on the guest's name or the booking reference; to open the detail of a reservation; to create a walk-in reservation for a guest who arrives without one; to assign specific physical rooms to a reservation; to reassign or unassign a room; to change the room type of a reservation subject to availability; to check a guest in; to check a guest out, including an early check-out recorded with the guest's consent; to cancel a reservation with a recorded reason, including a no-show; and to record a payment against a reservation.

The administrator requires the ability to manage rooms and room types, including their pricing, occupancy, features, images and overstay surcharge tiers; to manage staff accounts and to change a user's role; to edit the hotel's settings and stay policy; to list guests, payments and bookings; to restore a cancelled reservation; to view a dashboard and a reports view; and to review the activity log. The administrator is the only role that may reach these operations.

The system requires authentication, authorisation by role, validation of input at the interface, consistent computation of availability and price, idempotent creation of bookings, consistent error reporting, an audit log of important operations, and the delivery of transactional email and in-application notifications. Each of these is a requirement in its own right because it applies across the actors and cannot be attributed to one of them.

3.1.4.2 Non-Functional Requirements

The non-functional requirements concern the qualities that the system must exhibit. Reliability requires that inventory-changing operations be transactional and idempotent, so that a repeated request cannot create a duplicate booking and a concurrent request cannot exceed capacity. Security requires that permissions be enforced on the server, that passwords and verification codes be stored as hashes, that tokens be short-lived, and that responses not disclose whether an account or an email address exists (OWASP Foundation, 2021).

Maintainability requires a layered backend with enforced dependency directions and a typed frontend, so that a change in one area has a limited effect elsewhere. Usability requires a consistent interface for the public site and the staff consoles, with clear feedback for every action. Consistency requires that availability and price be computed once on the server and shared by every client. Efficiency requires that list operations be paginated and filtered rather than returning an entire table, and portability requires that the application run on the standard .NET and PostgreSQL toolchain without proprietary components. Finally, the non-functional

requirements include the constraint that no secret or payment credential be committed to source control, a requirement whose importance emerged during the work and is discussed in Chapter Four.

3.1.5 System Architecture

3.1.5.1 Architecture Overview

The system is a client-server web application. A React single-page application provides the user interface and communicates with an ASP.NET Core backend over HTTP using JSON, and the backend persists its state in a PostgreSQL database. The backend follows a representational style in which resources such as room types, bookings and payments are addressed by uniform resource identifiers and are acted upon with standard HTTP methods (Fielding, 2000). Every response, including every error, uses the Problem Details format with a machine-readable code, so that the frontend can react to a failure precisely rather than by parsing a message (Nottingham & Wilde, 2016). The public booking site, the receptionist console and the administrator console are three presentations of the same backend, and the backend, not the client, decides what each caller is allowed to do.

The two applications are deployed separately. The backend is a versioned ASP.NET Core service that exposes its contract under the path api/v1, and the frontend is a static single-page application that is served independently and calls that contract. Because the contract is versioned and documented, either side can be changed without breaking the other as long as the contract is respected, and a future change can be introduced under a new version while the current one continues to serve existing clients. The system is owned and hosted by the hotel rather than rented as a subscription, which keeps the hotel's records under its own control.

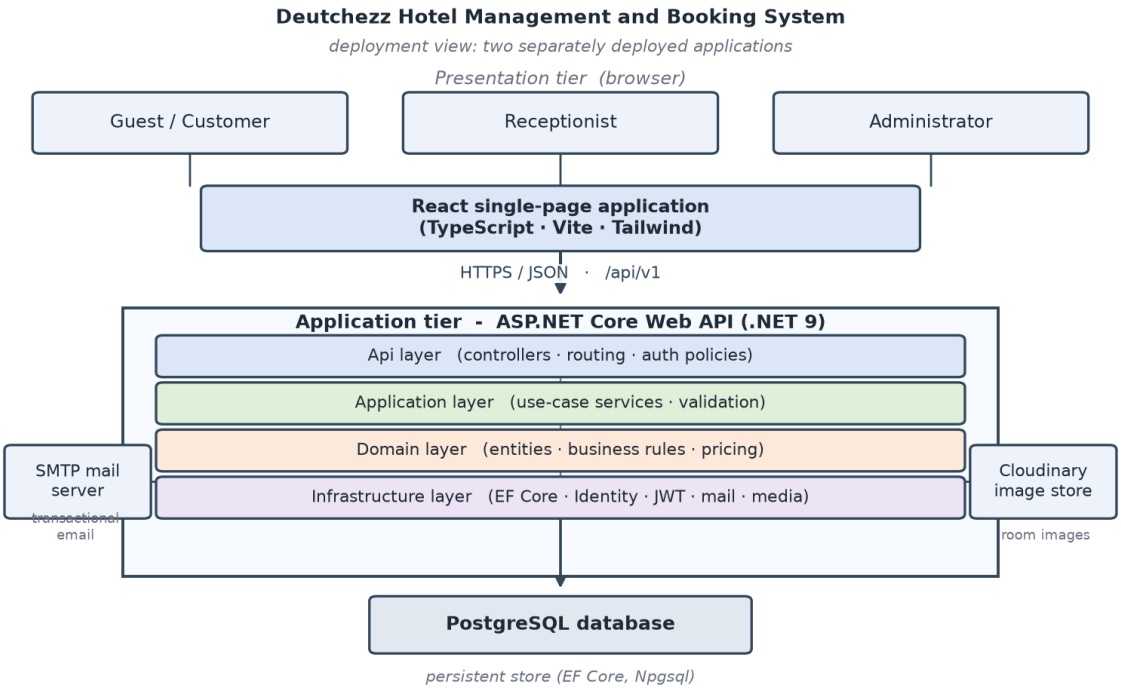


Figure 3.1: High-level client-server architecture

3.1.5.2 Backend Layering

The backend is organised into four logical layers that are kept in a single deployable web project for simplicity. The presentation layer holds the controllers and translates HTTP requests into application calls, the application layer holds the use-case services and the business rules, the domain layer holds the entities and enums that model the hotel, and the infrastructure layer holds the data access, the identity implementation, the email sender, the image storage and the other technical concerns. This is the layering that Sommerville (2016) describes as architectural design, in which the principal structural components and their relationships are identified and the allowed dependencies between them are fixed.

The dependency rule is that the domain depends on nothing, the application depends on the domain, and the infrastructure depends on the application, so that the direction of the arrows points inward and the domain remains free of technical concerns. The rule is not merely

documented: it is enforced by three automated architecture tests, so that a change which introduces a forbidden dependency fails the build rather than being discovered later in review. The layering keeps HTTP concerns, business rules, the domain model and the technical infrastructure separate from one another, so that each can be changed with a limited effect on the others (Gamma et al., 1994).

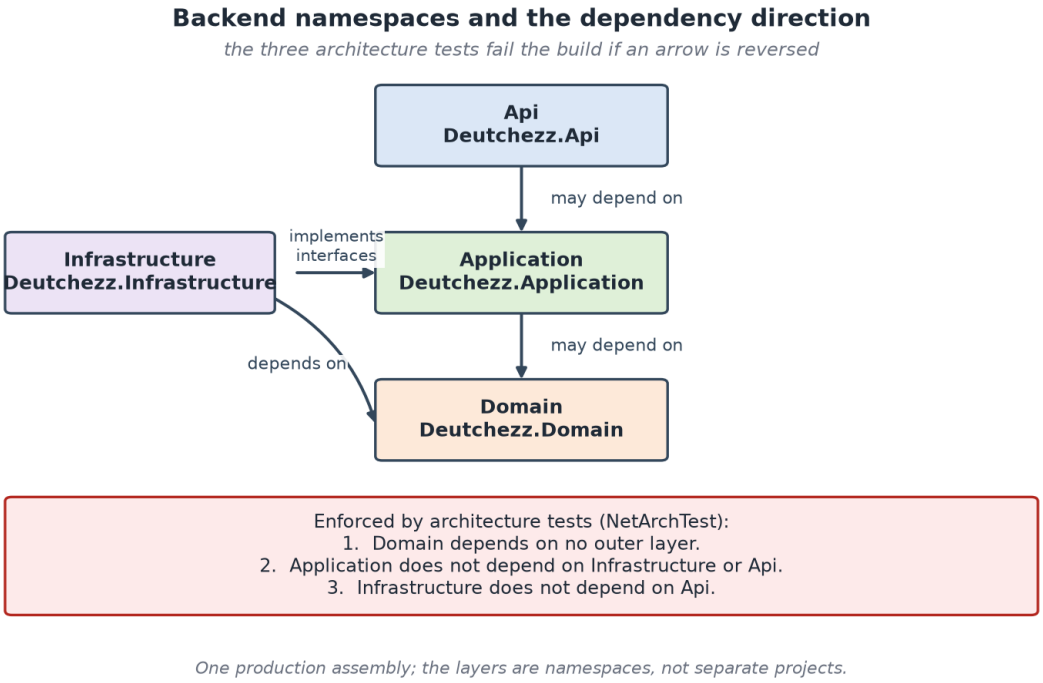


Figure 3.2: Backend layer structure and allowed dependencies

3.1.5.3 Request Lifecycle and Cross-Cutting Concerns

A request passes through a pipeline before it reaches the application logic. Authentication validates the bearer token and establishes the caller's identity and role; authorisation applies the policy that the endpoint declares; rate limiting restricts how often a caller may call the sensitive endpoints in a fixed window; and a global exception handler converts an unhandled error into a consistent Problem Details response rather than allowing it to escape. Because these concerns are applied by the framework, they are defined once and apply to every request, and a controller does not repeat them.

The application layer returns a result object that carries either a value or a structured error, and the presentation layer maps that result to the appropriate HTTP status code, which keeps the business rules independent of HTTP. This arrangement means that a validation failure, a conflict, an authentication failure and a permission failure are each reported in a single, predictable way, so that the frontend can present a specific message and can distinguish a validation failure from a conflict (Nottingham & Wilde, 2016). The uniform handling of errors is what allows the interface layer to remain thin and the contract to remain predictable.

3.1.6 Use Case Modelling

3.1.6.1 Actors

The system has three human actors and two supporting actors. The human actors are the customer, the receptionist and the administrator, and the supporting actors are the authentication subsystem and the business clock with its background workers. The responsibilities of each actor are summarised in Table 3.1.

Actor	Description	Principal use cases
Customer	A guest who books online	Browse room types; search availability; view a price quote; register and verify email; create, look up and cancel a reservation; manage own profile and notifications
Receptionist	A member of the hotel's front-desk staff	View and search bookings; create a walk-in booking; assign rooms; check in and out; produce a check-out quote; change room type; cancel a booking; record a payment
Administrator	The owner or manager of the hotel	Manage rooms and room types; manage staff and roles; edit settings and stay policy; view dashboard, reports, guests, payments and the activity log
Identity provider (supporting)	The authentication subsystem	Validate credentials; issue and rotate tokens; deliver verification and reset codes by email

Automated clock (supporting)	The business clock and background workers	Resolve local time; expire stale records; adjust booking statuses

Table 3.1: Actors and their principal use cases

The customer interacts with the system through the public website and, once registered, through a signed-in account. The receptionist and the administrator interact with the system through separate consoles, each of which exposes only the operations that its role is permitted to perform. The two supporting actors do not initiate use cases of their own; they act as the system carries out the use cases that need them, such as verifying a credential or deciding whether a pending reservation has expired.

3.1.6.2 Use Case Descriptions

Each use case was described by its actor, its trigger, its main success flow and its important alternative flows, and a representative selection is described here. The booking use case begins when a customer searches availability for a date range, continues when the customer selects a room type and confirms the reservation, and ends with a booking reference that is delivered to the customer. Its main alternative is insufficient availability, in which case the system returns a structured conflict rather than a partially created booking, and a second alternative is the detection of a possible duplicate of an earlier booking, which the customer must acknowledge before the reservation proceeds.

The check-in use case begins when a receptionist opens a confirmed booking, requires that at least one room has been assigned, and marks the booking as being in house while marking the assigned rooms as occupied. Its main alternative is an unpaid balance when the hotel has configured check-in to require full payment, in which case the operation is refused with a structured conflict. The check-out use case computes any overstay charge, records the final amount against the booking, releases the assigned rooms and marks the booking as complete;

its alternative is an early check-out, which requires the consent of the guest and is recorded with a reason.

The walk-in use case begins when a guest arrives without a reservation and the receptionist records the guest's details, the room type and the quantity at the desk. Its distinctive feature is that it creates a reservation for a guest who has neither an account nor an email address, so that the confirmation is sent to the member of staff rather than to the guest. The registration use case requires that the customer's email address be verified with a one-time code before the account is created, and its main alternative is an expired or incorrect code, in which case the account is not created and the code may be requested again.

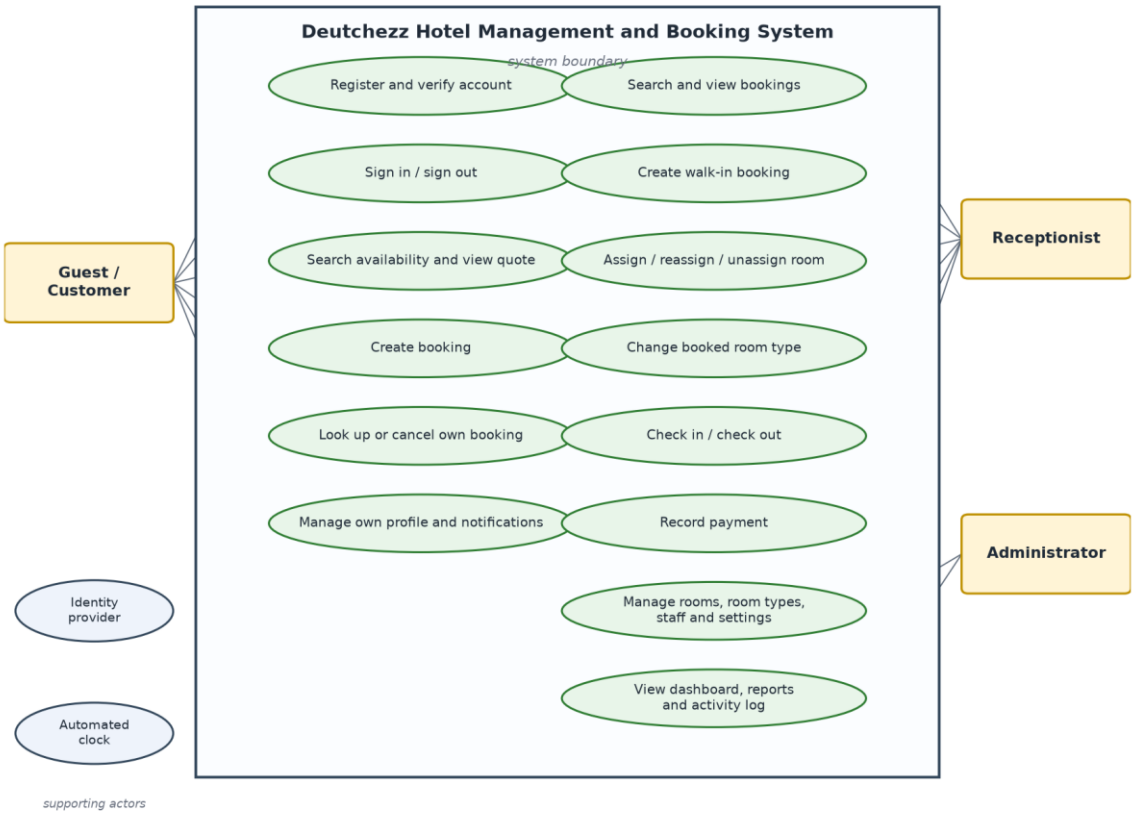


Figure 3.3: Use case diagram for the customer, receptionist and administrator actors

3.1.6.3 Role-Based Access Control

Access is controlled by three named policies, namely a customer policy, a staff policy that admits receptionists and administrators, and an administrator-only policy. Each endpoint declares the policy it requires, so that, for example, the room and staff administration endpoints are administrator-only while the reception console endpoints admit any member of staff. The public endpoints, such as room-type browsing, the availability search and the price quote, are explicitly anonymous, and the account and booking endpoints are scoped to the authenticated customer so that one customer cannot read another's reservations.

The policy is attached to the endpoint in the code, and the response returned to a caller who lacks the required role is a structured refusal rather than a partial result. This design follows the OWASP guidance that access control must be enforced on the server for every request rather than hidden in the client (OWASP Foundation, 2021). The consequence is that a

customer cannot reach a staff or administrative operation even by constructing a request directly, because the decision is made by the server and not by the interface.

3.1.7 Database Design

3.1.7.1 Entity-Relationship Model

The database is relational and was modelled with Entity Framework Core. It contains fifteen entity types, of which the central ones are the room type, the room, the booking, the booking line, the guest, the payment, the user, the room assignment and the setting. A room type has many rooms and many surcharge tiers; a guest has many bookings; a booking has many booking lines and many room assignments, and each line and each assignment belongs to one booking; a booking has many payments; and a room assignment relates one booking line to one room and records when the room was assigned and released.

The relationships express the three concerns that the design distinguishes. A booking belongs to a guest and carries one or more booking lines, each of which names a room type, a quantity and the nightly rate that was snapshotted at the time of booking. A booking also carries zero or more room assignments, each of which links one booking line to a physical room and records the instant at which it was made and, later, the instant at which it was released. Payments are attached to the booking rather than to a line, because a payment settles the reservation as a whole. This model allows the system to answer the availability question from the booking lines and the assignments, and to answer the reconciliation question from the payments.

The stay of a booking is stored as two UTC instants that mark the intended check-in and check-out moments, together with the two business dates that the guest selected. Storing the instants as well as the dates makes a reservation unambiguous regardless of the time zone of the caller, and it allows the duration-based price to be computed from the same instants that

the front desk uses. The business dates are resolved in the hotel's own time zone of Africa/Lagos, so that a booking made at one minute past midnight local time belongs to the correct local day.

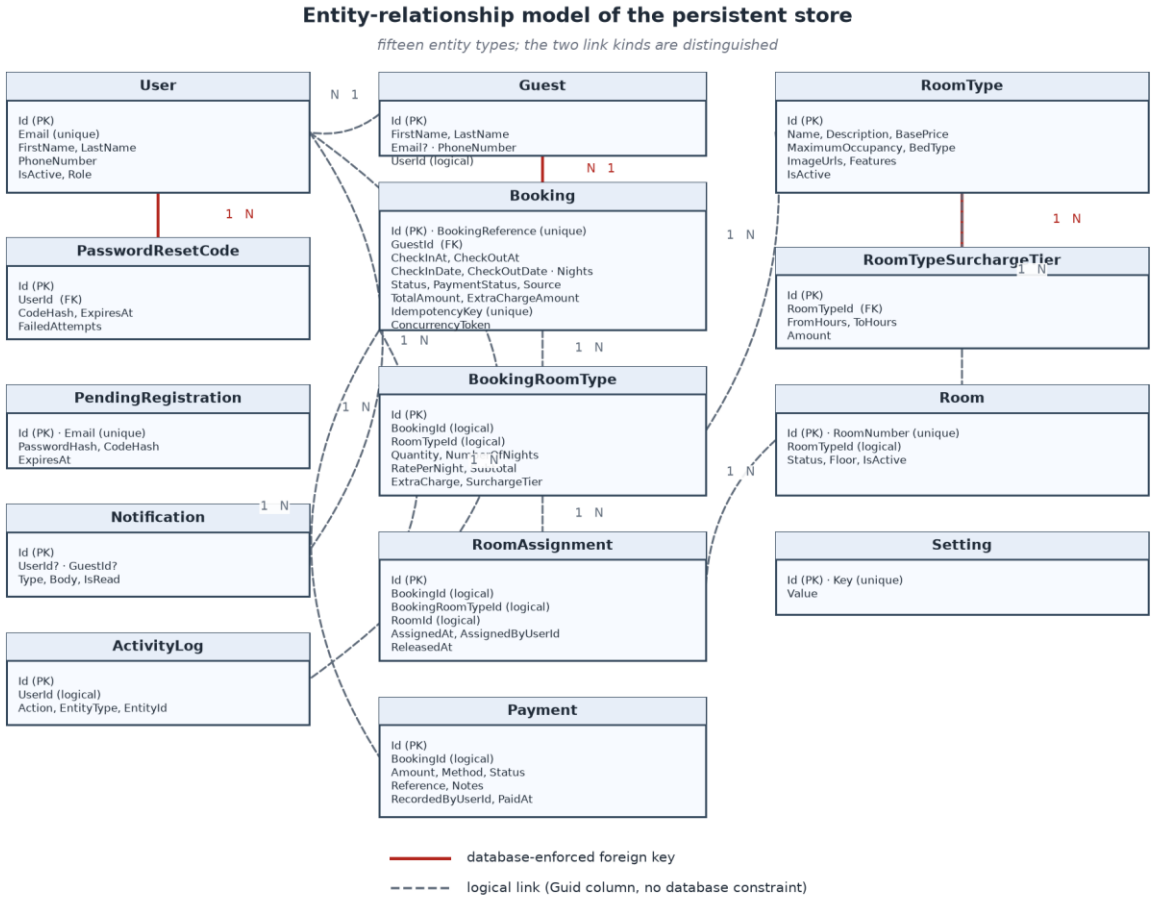


Figure 3.4: Entity-relationship diagram

3.1.7.2 Data Dictionary

Table 3.2 summarises the entities of the model and their principal attributes, and Table 3.3 lists the enumerations that constrain the values of several attributes.

Entity	Purpose	Key attributes
RoomType	A class of room offered by the hotel	Name; Description; BasePrice; MaximumOccupancy; BedType; ImageUrls; Features; IsActive
RoomTypeSurchargeTier	A fixed-amount overstay band for a room type	RoomTypeId; FromHours; ToHours; Amount
Room	A physical room	RoomNumber; RoomTypeId; Status; Floor; IsActive

Booking	A reservation held by a guest	BookingReference; GuestId; CheckInAt/CheckOutAt; CheckInDate/CheckOutDate; Nights; Adults; Children; Status; PaymentStatus; Source; TotalAmount; ExtraChargeAmount; IdempotencyKey; ConcurrencyToken
BookingRoomType	A line of a booking	BookingId; RoomTypeId; Quantity; RatePerNight; NumberOfNights; Subtotal; ExtraCharge; SurchargeTier
RoomAssignment	The link between a booking line and a room	BookingId; BookingRoomTypeId; RoomId; AssignedAt; AssignedByUserId; ReleasedAt
Guest	A person who holds bookings	FirstName; LastName; Email; PhoneNumber; UserId
Payment	A payment recorded against a booking	BookingId; Amount; Method; Status; Reference; Notes; RecordedByUserId; PaidAt
User	An account that can sign in	Identity fields; FirstName; LastName; IsActive; LastLoginAt; RefreshTokenHash; RefreshTokenExpiresAt
Role	The role assigned to a user	Identity role fields
Setting	An editable hotel setting	Key; Value
Notification	A message for a user or guest	UserId; GuestId; Type; Body; IsRead
ActivityLog	An audit entry	UserId; Action; EntityType; EntityId; Description; CreatedAt
PendingRegistration	An unverified registration awaiting its code	Email; PasswordHash; CodeHash; CodeExpiresAt; FailedAttempts; SendCount; ExpiresAt
PasswordResetCode	An active password-reset code	UserId; CodeHash; ExpiresAt; FailedAttempts; SendCount; UsedAt

Table 3.2: Summary of the database entities

Enumeration	**Values**
BookingStatus	Pending; Confirmed; CheckedIn; CheckedOut; Cancelled
RoomStatus	Available; Occupied; Maintenance; Inactive
PaymentStatus	Pending; PartiallyPaid; Paid; Refunded; Failed
PaymentMethod	Cash; POS; Bank transfer; Online

BookingSource	Online; Walk-in
UserRole	Customer; Receptionist; Administrator
NotificationType	BookingConfirmed; BookingCancelled; PaymentRecorded; CheckInReminder; CheckOutReminder

Table 3.3: Enumerations used by the system

The seven enumerations are deliberately small. The booking status follows the lifecycle of a reservation from its creation to its completion or cancellation, and every ordinary transition is permitted in one direction only. The room status describes the physical state of a room, which is distinct from its future availability, because a room that is occupied today may be free for a future date and a room that is free today may be held for a future date. The payment status records whether a payment has been made, refunded or has failed, and the payment method records how it was made. The source distinguishes a reservation created by a guest online from one created by the front desk, and the notification type distinguishes the transactional messages that the system can raise.

Money is stored as a decimal of fixed precision and scale and is rounded to two places at each step, and the business rules of the system state that a booking's total amount is the sum of its line subtotals with no tax, service charge or fee added, because the hotel has not asked for one. Client-submitted prices and totals are ignored, so that the amount charged cannot be altered by a caller who edits a request before sending it.

3.1.7.3 Migrations and Schema Evolution

The schema is created and evolved through ordered migrations rather than by editing the database by hand. Nine migrations were applied over the life of the project. They established the initial schema, added the display fields of a room type and the overstay tiers, introduced the user, guest, booking, payment and setting records, added the pending-registration and password-reset records, added the notification and activity-log records, added the room-

assignment record, and finally added the duration-based pricing tier of the current version. Each migration records one change together with the code that depends on it.

Keeping the schema in migrations means that a new environment can be brought to the current shape by applying the history in order, and that a change to the data model can be reviewed and, if necessary, reversed, rather than being applied by hand to a live database. The same migration history is applied to the development and test databases, and the test database is recreated for each integration run, so that a test never depends on data left behind by an earlier one.

3.1.8 Data Flow and Process Modelling

Data flow through the system in a predictable pattern. A request is authenticated and authorised, its payload is validated, and the application layer performs the work inside a transaction wherever the operation changes inventory. An availability search reads the active rooms of each requested room type and excludes those that a conflicting reservation or assignment has taken for the requested instants, and it returns the remaining quantity together with a price computed by the same service that will compute the price of the booking. A booking creation re-reads availability under a lock, writes the guest, the booking, its lines and its initial state, and proceeds only if the capacity is still sufficient at the moment of writing.

The reception operations follow the same pattern as the booking flow. An assignment locks the room, checks that it is free for the whole stay and creates the assignment; a check-in verifies that an assignment exists and moves the booking into the in-house state; a check-out computes any overstay charge, records it and releases the assignments; and a payment is recorded against the booking and updates its payment status. Administrative writes update the room, the room type or the setting and record an activity-log entry, so that every change to the configured data is auditable.

Two background processes run outside the request cycle. The first expires pending registrations that have not been verified within the configured interval, so that abandoned registrations do not accumulate. The second adjusts the status of reservations as time passes, so that a reservation that is no longer held does not consume availability. Both are implemented as background services that run on a schedule and are idle when there is nothing to do.

Figure 3.5 presents the booking process as an activity diagram. It shows the request entering the system, the idempotency pre-check that returns the earlier booking when a request is replayed, the validation of the stay dates and the guest, and the transaction boundary within which the room-type rows are locked, availability is re-computed, the booking and its lines are written and the transaction is committed. The two early exits, the validation failure and the insufficient-availability conflict, are drawn to the side so that it is clear that neither leaves a partially created booking behind. Figure 3.6 then shows the same process as a sequence diagram, in which the customer, the frontend, the booking controller, the booking service, the availability service and the database exchange messages in time order, and it makes explicit that the availability check is performed under the same lock that protects the write.

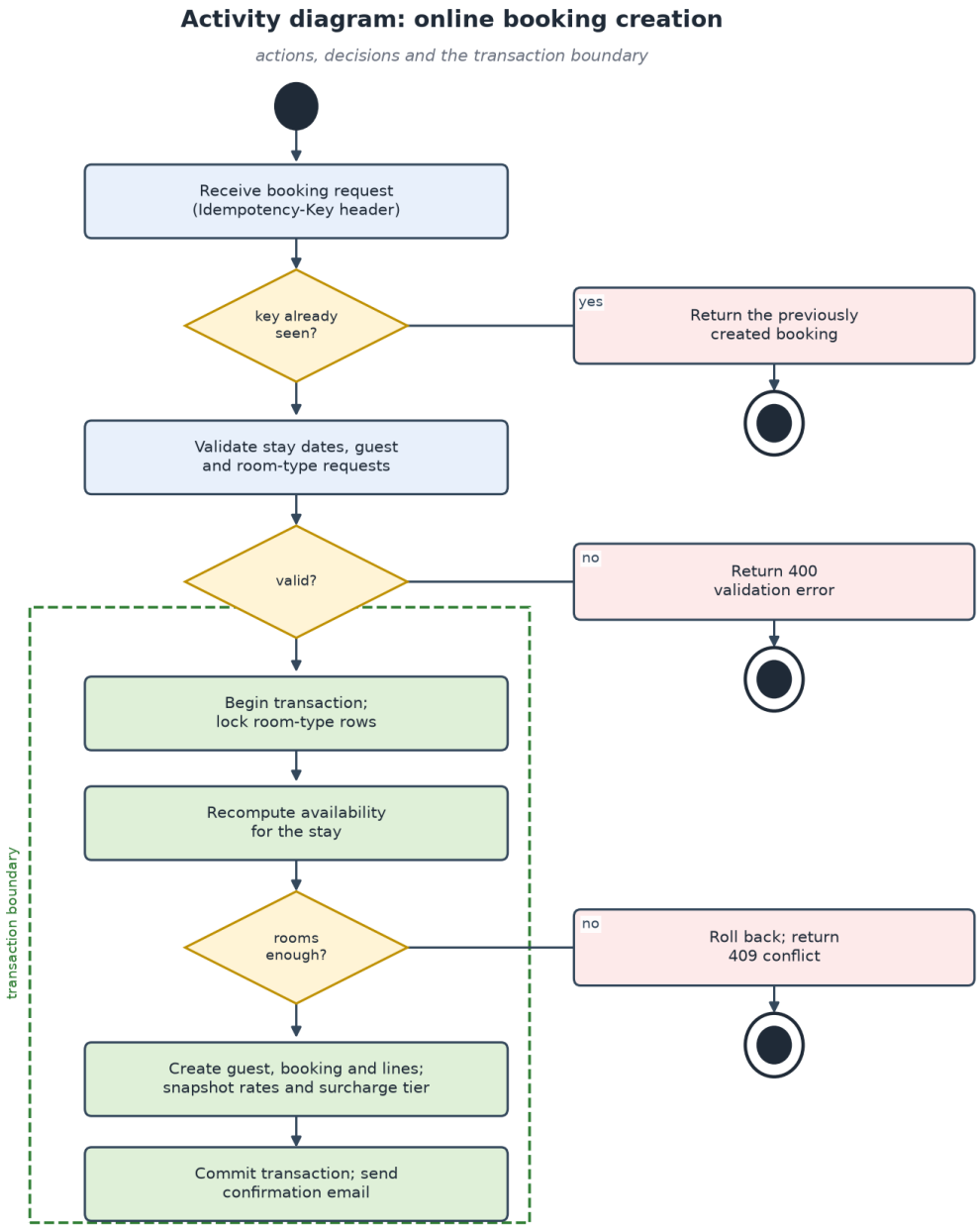


Figure 3.5: Activity diagram for the online booking process

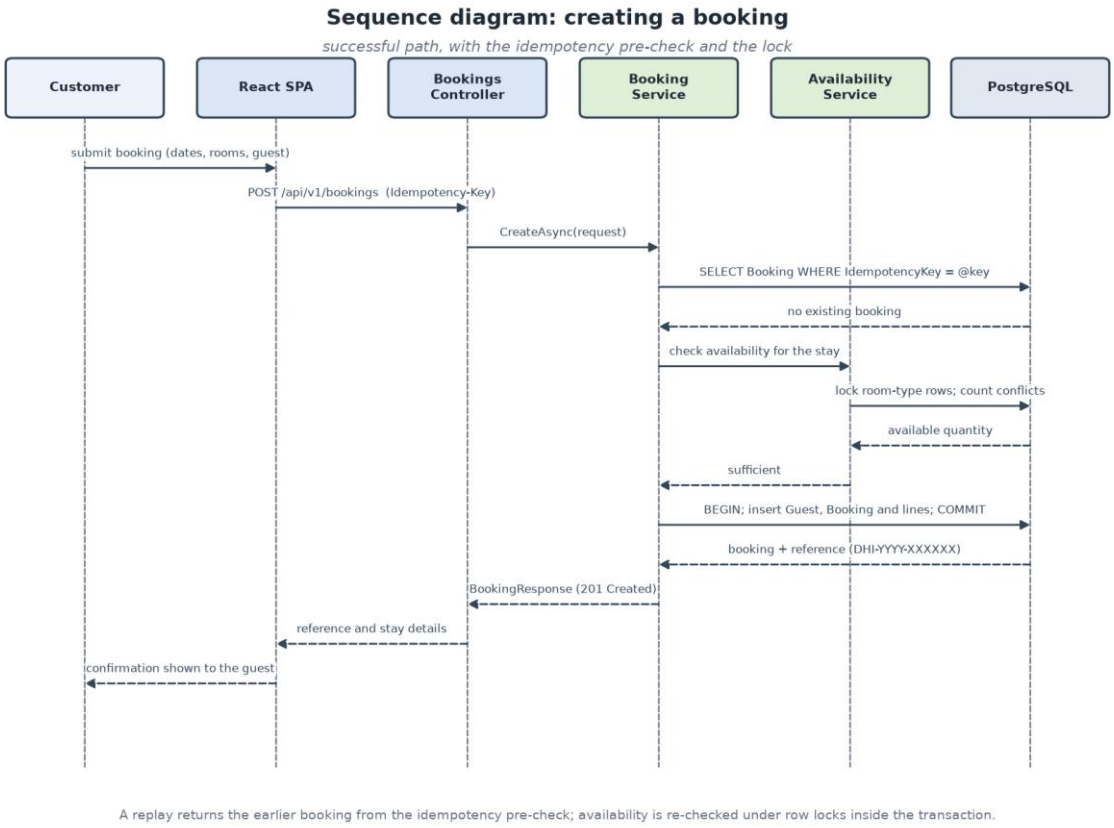


Figure 3.6: Sequence diagram for booking creation

3.1.9 System Modules

The system is divided into modules that correspond to the areas of the hotel's operation and to the groups of endpoints that the backend exposes. Table 3.4 lists these modules and describes the responsibility of each.

Module	**Description**
Room-type catalogue	Publishes the hotel's room types, their images, features and prices, and allows an administrator to maintain them
Availability and pricing	Computes available quantities for a date range and returns a consistent price quote for a stay
Booking	Creates online and walk-in reservations, assigns a reference, and exposes lookup and cancellation
Registration and authentication	Handles account creation with email verification, sign-in, token rotation and password reset
Customer account	Provides a signed-in customer's bookings, profile and notifications
Reception console	Supports booking search, room assignment,

	check-in, check-out, room-type change and payment recording
Administration	Provides room, room-type, staff, settings, guest, payment, report and activity-log management
Notifications	Delivers transactional email and in-application notifications
Background maintenance	Runs scheduled work such as expiring stale registrations and adjusting booking statuses
Audit	Records the important operational and administrative actions

Table 3.4: System modules

The modules are not separate deployable units; they are logical groupings of endpoints and services that share the same layered backend. Their value is that they make the responsibility of each part of the system explicit, and they allow the tests to be organised by the area they exercise. A change that affects availability or price is made in the availability-and-pricing module, and a change to the way a room is published is made in the room-type catalogue, so that the effect of a change is bounded by the module that owns it.

3.2 Technology Stack

The technologies were selected for maturity, type safety and cost, and each was chosen because it supports a requirement of the design rather than because it is fashionable. They are listed in Table 3.5.

Layer	**Technology**	**Reason**
Backend	C# on .NET 9 with ASP.NET Core	Strong typing, mature web framework and cross-platform hosting
Data access	Entity Framework Core	Object-relational mapping with migrations and transaction support
Database	PostgreSQL	Reliable relational database with row-level locking
Identity	ASP.NET Core Identity with JSON Web Tokens	Standard credential handling with stateless access tokens
Frontend	React with TypeScript and Vite	Component-based user interface with static typing and a fast build
Styling	Tailwind CSS	Consistent, utility-based

		styling
Image storage	Cloudinary	Hosted storage and delivery of room images
Testing	xUnit and FluentAssertions; Vitest	Automated backend and frontend testing
Version control	Git	History and branching of the source

Table 3.5: Technology stack

The backend was written in C# on .NET 9 with ASP.NET Core because the framework provides a mature and strongly typed web host, dependency injection, authentication and authorisation, and a rich data-access story through Entity Framework Core. The strongly typed language reduces a class of errors that would appear only at run time in a dynamically typed language, and the type system is used to express the domain model and the contracts. Entity Framework Core was chosen as the object-relational mapper because it supports migrations, transactions and the concurrency facilities that the design depends on, and PostgreSQL was chosen as the database because it is a reliable open-source relational database with the row-level locking facility that the availability check requires (PostgreSQL Global Development Group, n.d.).

Identity is handled by ASP.NET Core Identity together with JSON Web Tokens, which gives a standard implementation of credential storage and a stateless access token that the frontend can present on each request. A refresh token is used to obtain a new access token without asking the guest to sign in again, and it is stored only as a hash. The frontend was written in React with TypeScript and built with Vite, because the component model suits the interactive interfaces of the booking flow and the two consoles, and because static typing allows the frontend to be checked against the shape of the API contract. Tailwind CSS provides a consistent set of styles, Cloudinary provides the hosted storage and delivery of room images, and Vitest provides the frontend test runner. Git was used for version control so that each increment could be reviewed and, if necessary, reversed.

The selection is deliberately conservative. Each technology is widely used and supported, the backend and the frontend are both strongly typed, and every part of the stack is freely available, so that the hotel is not committed to a licence or to a hosted service beyond the optional image storage. The cost of the stack is the learning required to work in it, which is a one-off cost, rather than a recurring subscription.

3.3 Implementation

This section describes the decisions that shaped the implementation and that are reported in Chapter Four. It covers the lifecycle of a booking, the way availability is computed and protected, the way booking creation is made idempotent, the way the duration-based price is calculated, the way input is validated and errors are reported, and the way security is enforced.

3.3.1 Booking Lifecycle and State Transitions

A booking passes through a small set of states. In the ordinary flow a reservation is created pending, is confirmed when the hotel accepts it, becomes checked in when the guest is given a room and becomes checked out when the stay ends; a reservation may instead be cancelled before it reaches its end. Because the hotel takes payment at the desk, a reservation created through the current system is created directly in the confirmed state and is not subject to the pending-expiry process, and the pending state is retained for a future online-payment flow. The backend refuses any illegal transition, such as moving a completed booking back to an earlier state.

Each operation that changes the state of a booking also records the change. A check-in stores the actual check-in instant, a check-out stores the actual check-out instant, and a cancellation stores the instant and the reason. The activity log receives an entry for each of these operations, carrying the actor, the action and the booking reference, so that the history of a

reservation can be reconstructed after the fact. This is the audit trail that the analysis of the existing system found to be missing, and it is what allows a disputed reservation or payment to be examined against the sequence of operations that produced it.

The state model also governs which operations are permitted. A room may be assigned only while a booking is confirmed or in house, an unassignment is allowed only before check-in, and a cancellation is refused for a booking that is already in house or complete. The rules are enforced on the server rather than in the interface, so that the same state machine governs every client, and a request that would move a booking through an illegal transition is answered with a structured conflict rather than a silent change.

3.3.2 Availability and Concurrency

The prevention of overbooking is the central reliability problem of the system, and it is solved by taking database locks in a consistent order inside the transaction that creates or changes a booking. When a booking is created, the system locks the relevant room-type rows, re-computes the available quantity against the requested stay, and proceeds only if the quantity is still sufficient (PostgreSQL Global Development Group, n.d.). Because the locks are taken in ascending identifier order, two concurrent transactions cannot deadlock against each other, and the second transaction observes the effect of the first.

Availability itself is computed from a half-open interval. An existing reservation conflicts with a requested stay when its check-in is before the requested check-out and its check-out is after the requested check-in, so that a guest who leaves on the tenth does not block a guest who arrives on the tenth, and a room can be re-let on the day of a departure. The available quantity of a room type for a range is the number of active rooms of that type less the quantity held by the reservations that overlap the range, and it is never allowed to fall below zero. The rule is implemented once, in a single service, so that the availability reported by a

search, the availability checked at booking and the availability shown at assignment cannot disagree.

Alongside the pessimistic lock the design uses an optimistic concurrency token on the booking entity, which detects a lost update on any path that does not take the lock (Microsoft, 2023). The two mechanisms are used together because they answer different questions: the lock prevents two availability-affecting operations from proceeding at the same time, and the token detects a change that was made to a record between the moment it was read and the moment it was written. The choice follows the guidance that the concurrency strategy should match the cost of an error, which for a booking is high (Kleppmann, 2017).

3.3.3 Idempotent Booking Creation

A booking is created only once even if a client retries the request. The client supplies an idempotency key with the request, the server hashes the request body and stores both the key and the hash against the booking, and a unique database index prevents a second booking with the same key. If the same key is presented again with the same body, the original booking is returned rather than a duplicate; if the same key is presented with a different body, the request is rejected as a conflict. The identity of the request is therefore the pair of the key and the body, and neither half alone is sufficient.

This design makes the booking operation safe to retry after a network timeout, which is a common failure in a public web application (Kleppmann, 2017). A guest who submits a booking and then loses the network before receiving the confirmation can retry with the same key and receive the same booking rather than a second one, and a client that reuses a key with a changed body is told that the request does not match the original rather than being silently served. The unique index makes the guarantee a property of the database rather than

of the application alone, so that it holds even if two requests with the same key arrive at the same instant.

3.3.4 Duration-Based Pricing

The pricing model bills each booked calendar night at the nightly rate of the room type and treats any time beyond those nights as an overstay. The base charge is the nightly rate multiplied by the number of booked nights and the quantity, and the overstay is the time between the booked check-out moment and the actual check-out moment. An overstay within a grace window is not charged, so that a guest who leaves shortly after the booked hour is not penalised; an overstay beyond the grace window is charged the fixed amount of the matching tier for that room type; and an overstay beyond the last tier is charged one further full nightly rate.

The price is calculated by a single service and is exposed through a quote endpoint, so that the figure shown to a guest before booking and the figure used when the booking is created come from the same code. The tiers of a room type are validated to be contiguous and ordered, so that every duration beyond the grace window falls into exactly one tier, and the applied tier is snapshotted onto the booking line so that a later change to the tier table does not alter the amount of a reservation that has already been made. This snapshotting is what keeps historical amounts stable when the hotel adjusts its overstay policy. Table 3.6 states the rule in full.

Element	**Rule**
Base charge	Nightly rate multiplied by booked nights multiplied by quantity
Grace window	An overstay within the window is not charged
Tier charge	An overstay beyond the window is charged the matching fixed tier amount
Beyond the last tier	One further full nightly rate is charged
Snapshot	The applied tier is stored on the booking line so later table changes do not alter history

Table 3.6: The duration-based pricing rule

3.3.5 Validation and Error Handling

Input is validated at the boundary, and the stay rules are resolved by a dedicated service that enforces the ordering of the check-in and check-out moments, their permitted ranges and, for a request made by a guest rather than by staff, that the check-in is not in the past. The rules of a booking are checked together: the check-in must not be before the current day in the hotel's time zone, the check-out must be strictly after the check-in, the number of nights must be at least one, the number of adults must be at least one, the number of children must not be negative, the total number of rooms must lie between one and the configured maximum, every room type must exist and be active, the guest's email must be well formed and a telephone number must be present.

Every failure is returned as a Problem Details response with a stable, machine-readable code, so that the frontend can present a precise message and can distinguish, for example, a validation failure from a conflict (Nottingham & Wilde, 2016). A validation failure is reported with a status of four hundred, an authentication failure with four hundred and one, a permission failure with four hundred and three, a missing record with four hundred and four, a conflict with four hundred and nine and a rate-limited request with four hundred and twenty-nine. The status codes are fixed by the convention, so that the same situation is always reported in the same way.

The JSON conventions are equally fixed. Property names are written in camel case, enumeration values are serialised as their names rather than as numbers, dates are written as a year-month-day string and money as a decimal number. Every list endpoint returns a paged envelope that carries the items together with the page number, the page size, the total count and the total number of pages, and it accepts a sort field and a direction. Fixing these

conventions means that a client can be written against any endpoint with confidence, and the contract is documented in the project's source of truth so that the client and the server are built to the same agreement.

3.3.6 Security Implementation

Passwords are hashed with the algorithm provided by the identity framework, and a refresh token is stored only as a hash with an expiry, so that a stolen token cannot be replayed after rotation. Registration and password reset use a six-digit code that is derived with a keyed hash and compared with a constant-time comparison, and these codes are rate limited and locked after repeated failures. Responses for a forgotten-password request are deliberately identical whether or not the address exists, which prevents the endpoint from being used to discover which accounts exist. These controls implement the authentication and access-control recommendations reviewed in Chapter Two (OWASP Foundation, 2021).

The design treats access control as a server-side decision rather than a client-side convenience. Each endpoint declares the policy that it requires, the policy is evaluated on the server for every request, and the result is a structured refusal when the caller is not permitted. The combination of a hashed password, a rotating refresh token, a keyed and constant-time verification code and an identical response for a forgotten-password request means that the most direct attacks on an account, namely guessing the password, replaying a stolen token, guessing the code and enumerating the accounts, are each answered by a specific control rather than by a general assurance.

One weakness was identified outside the code and is recorded here so that it is not lost. During development, configuration files that were committed to the repository were found to contain third-party credentials and a mail password. The values are not reproduced in this report, but they must be treated as exposed and rotated, and future configuration must be

supplied through user secrets or environment variables rather than committed. The finding is stated as a limitation of the working environment rather than of the design, and it is carried into the recommendations of Chapter Five.

3.4 Testing Methodology

The testing methodology combines automated tests at four levels with manual functional testing of the user journeys. The automated tests are organised by the layer they exercise, so that a failure identifies the layer in which it occurred, and they are run during development rather than only at the end. Table 4.2 of Chapter Four reports the suites and the outcome of the runs that were possible.

3.4.1 Unit Testing

Unit tests cover the pure logic of the domain and the application, including the pricing formula and its tier boundaries, the stay-policy rules and the value rules of the entities. These tests are fast and independent of the database, and they were written alongside the code they exercise. Because the pricing and stay rules are the parts of the system where a boundary error is most costly, they carry a larger share of the unit tests than their size would suggest, and the tests include the exact boundaries between the tiers and the grace window.

3.4.2 Integration Testing

Integration tests exercise the API against a real database across the availability, booking, reception and administration flows, and they verify the concurrency and idempotency behaviour that unit tests cannot reach. A test of this kind creates a booking, attempts a conflicting booking and asserts that the second is refused, and it repeats a booking with the same idempotency key and asserts that a single reservation results. The suite requires a running PostgreSQL instance; that instance was not available in the environment used for

this report, so the integration suite was not executed here and the report makes no claim about its result.

3.4.3 Architecture Testing

Three architecture tests enforce the dependency rule described in Section 3.1.5.2, so that a change which introduces a forbidden dependency fails the build rather than being discovered later in review. The tests assert that the domain depends on no outer layer, that the application does not depend on the infrastructure, and that the presentation does not bypass the application to reach the domain or the infrastructure directly. These tests are cheap to run and they protect the layering that the rest of the design depends on.

3.4.4 Frontend Testing

The frontend has a suite of component and utility tests covering the booking flow, the pricing display and the account screens, together with static type checking and linting. These tests were run during development and are reported in Chapter Four. The type checking is treated as part of the test suite because it detects a class of contract mismatch between the frontend and the API before the application is run.

3.4.5 Functional Testing

In addition to the automated tests, the principal user journeys were exercised manually through the interface, namely browsing rooms, searching availability, booking as a guest and as a customer, verifying an account, checking in and out a booking at the reception console, recording a payment and reviewing the administration screens. The observations from this testing and the state of the remaining, unverified journeys are reported in Chapter Four. The manual testing is described as it was performed, namely by the developer against the running system, and it is not presented as an independent acceptance test.

3.5 Summary

This chapter has described how the system was analysed and designed, and the decisions that its implementation takes from that design. It set out the methodology and the data-collection approach, analysed the manual system and its weaknesses, and presented the proposed system with its advantages. It stated the functional and non-functional requirements, and described the architecture, the actors and use cases, the data model, the process model and the modules of the system. It set out the technology stack and the reasoning behind each choice. It then explained the principal implementation decisions, namely the booking lifecycle, the availability and concurrency design, the idempotent creation of bookings, the duration-based price, the validation and error-handling conventions and the security controls. It closed with the testing methodology and the four levels of automated test together with the manual functional testing. Chapter Four presents the implementation of this design and the evaluation of the resulting system.

CHAPTER FOUR

SYSTEM IMPLEMENTATION AND EVALUATION

4.0 Introduction

This chapter presents the implemented system and evaluates it against the objectives set out in Chapter One and the requirements set out in Chapter Three. It begins with an overview of the implementation and then describes the principal user interfaces and features of the public booking site, the customer account, the receptionist console and the administrator console. It describes the application programming interface that the interfaces are built on, then evaluates the system functionally, examines its reliability and security controls, and reports the results of the automated tests that were executed. The chapter closes by aligning the results with the objectives and research questions, comparing the system with the related works reviewed in Chapter Two, and stating the status of user acceptance testing. Where a result rests on a manual observation or on a test run, the report says so, and where evidence is unavailable the report says that rather than presenting an estimate.

4.1 System Implementation Overview

The system was implemented as two applications. The backend is an ASP.NET Core web application that hosts the four logical layers described in Chapter Three and exposes the versioned API under the path api/v1, and the frontend is a React single-page application written in TypeScript and built with Vite. The two applications are developed and deployed separately, so that the backend can be hosted on any platform that supports .NET and the frontend can be served as static files. The backend is the authority for availability, price and state; the frontend renders what the backend reports and submits requests to it, and it does not attempt to compute availability or price on its own. The data is held in PostgreSQL and is created and evolved through the nine migrations listed in Section 3.1.7.1.

The backend is arranged so that each area of the hotel's operation has a service of its own. The booking, availability, reception, administration, room-type, authentication, notification and email concerns are each implemented as a service that the controllers call, and the services depend on the domain model and on the abstractions of the infrastructure rather than on the infrastructure itself. This keeps the business rules in one place per concern and allows the interfaces of the infrastructure, such as the mail sender and the image store, to be replaced without changing the rules. The services are registered through the dependency-injection container that the framework provides, so that the dependencies of each service are declared once and resolved by the container rather than constructed by hand.

Configuration is read from a layered set of sources: the application settings, the environment, and the database-backed settings that an administrator can edit. The settings that the hotel adjusts, such as the maximum number of rooms in one booking, the check-in and check-out times and the cancellation window, are read through a single settings provider so that a caller does not depend on where the value came from. A value that is present in the database takes precedence over the application default, and the provider falls back to the application settings when the database has no row, so that the system behaves correctly before an administrator has changed anything.

The implementation is organised around the ten controllers listed in Table 4.1, which together expose sixty-six endpoint actions. Each controller groups the endpoints of one concern, and each endpoint declares exactly the access it requires, whether anonymous, customer-scoped, staff or administrator-only. Services are registered through the dependency-injection container, and cross-cutting behaviour such as authentication, authorisation, rate limiting and error handling is applied by the framework rather than repeated in each controller. This arrangement keeps each controller small and keeps the business rules in the application layer, which is what the architecture tests enforce.

Controller	Base route	Endpoint actions
Auth	api/v1/auth	9
Me	api/v1/me	8
Bookings	api/v1/bookings	3
Availability	api/v1/availability	1
Pricing	api/v1/pricing	1
Room types	api/v1/room-types	2
Stay policy	api/v1/stay-policy	1
Reception	api/v1/reception	15
Admin	api/v1/admin	25
Uploads	api/v1/uploads	1
Total		66

Table 4.1: API controllers and the number of endpoints in each

The allocation of endpoints among the controllers reflects their audience. The nine authentication endpoints support registration, verification, resending a code, sign-in, token refresh, forgotten and reset password, sign-out and the current user, and they are the only endpoints that a caller who is not signed in reaches apart from the public catalogue, the availability search and the price quote. The eight endpoints grouped under the signed-in customer serve the customer's own bookings, profile and notifications. The twenty-five administrative endpoints are the largest group because the administrator maintains the room inventory, the room types, the staff, the settings and the reporting and audit views. The fifteen reception endpoints are the operational set of the front desk, covering search, detail, walk-in creation, cancellation, assignment, room-type change, check-in, check-out and payment recording.

4.2 User Interfaces and Features

4.2.1 Room Discovery

The public site opens on the hotel's room types, each presented with its name, description, nightly price, maximum occupancy, bed type, features and images. The room-type data is served by the public room-type endpoints, which return only active room types, and the

images are hosted on a managed image service. The purpose of this feature is to let a prospective guest understand what the hotel offers before checking availability, and it corresponds to the first customer use case in Chapter Three. It is implemented as a list that leads to a detail view for a single room type.

The room-type endpoints are deliberately anonymous, because a guest must be able to see the hotel's offering before deciding whether to register. Only room types that are active are returned, so that a type the hotel has retired disappears from the public site without its record being deleted, and the images that accompany a room type are stored in the hosted image service and delivered from there rather than from the application host. An administrator maintains the images from the administration console described later in this section, and the public view reads them without any further processing. The listing is paginated by the shared list convention, so that the catalogue can grow without the page becoming slow.

[TO BE PROVIDED: insert screenshot image for Figure 4.1]

Figure 4.1: Room-type listing on the public site

[TO BE PROVIDED: insert screenshot image for Figure 4.2]

Figure 4.2: Room-type detail with images, features and price

4.2.2 Availability Search and Pricing Quote

A guest searches for a stay by choosing a check-in date, a check-out date and the number of adults and children, optionally with a room type and a time. The availability endpoint returns, for each room type, the number of rooms still available over the requested instants together with the price for the stay. The price is computed by the same service that computes the price at booking time, so the figure shown to the guest is the figure the booking will use. This addresses the research question on determining available rooms, and it is the feature that replaces the manual availability check. A guest who has not yet decided on dates can also request a price quote directly from the pricing endpoint.

The search applies the half-open overlap rule described in Chapter Three, so that a reservation ending on one day does not hide a room that becomes free on that day, and it counts only the reservations that consume availability. A room that is physically occupied today is still offered for a future date, because its current state is not the same as its future availability. The price that accompanies each result is produced by the pricing service and includes the booked nights for the requested range, so that the guest can compare room types on the basis of what a stay would actually cost rather than on the nightly rate alone. Both the search and the quote are anonymous, because they must be available to a guest who has not yet created an account.

[TO BE PROVIDED: insert screenshot image for Figure 4.3]

Figure 4.3: Availability search form and results with prices

4.2.3 Reservation and Booking Confirmation

When a guest confirms a selection, the booking endpoint creates the reservation and returns a booking reference in the form DHI-YYYY-XXXXXX. The booking stores the guest, the requested room types and quantities, the stay instants and dates, the number of nights and the total amount, and it is created in a single transaction that locks the relevant rooms and re-checks availability so that a concurrent request cannot overbook the hotel. The creation is idempotent: a repeated request with the same key returns the original booking rather than a duplicate. On success the guest is shown the reference together with the stay details, and a confirmation is sent by email. A guest can later look up the booking by its reference and cancel it if the hotel's policy permits.

The reference is generated on the server, and a guest who is not signed in is still able to look the reservation up with the reference and the email address that was used to make it, so that a booking is never lost merely because no account was created. The same endpoint serves a signed-in customer and an anonymous guest, and the system attaches the reservation to the

customer's account when the caller is signed in and otherwise records the guest on its own. When a reservation is found to resemble an earlier one made by the same guest, the server returns a structured warning that the guest must acknowledge before the reservation proceeds, which protects against the accidental double-booking that a repeated submission can cause.

[TO BE PROVIDED: insert screenshot image for Figure 4.4]

Figure 4.4: Booking form with the computed price

[TO BE PROVIDED: insert screenshot image for Figure 4.5]

Figure 4.5: Booking confirmation showing the reference and stay details

4.2.4 Registration and Email Verification

A new account is created in two steps. The visitor submits their details, and the system stores a pending registration together with a hashed password and a hashed six-digit code, and sends the code by email. The visitor enters the code, and only when it matches is the account created and the pending registration removed. The code is rate limited, expires after a fixed interval and is locked after repeated failures, and the comparison is performed in constant time. This design means that an account cannot be created for an email address that the person does not control, and it implements the authentication recommendations reviewed in Chapter Two (OWASP Foundation, 2021).

The separation between a pending registration and a confirmed account is what allows the account to be created only after the address has been proven. A pending registration that is never verified does not become an account, and a background process removes it after the configured interval so that abandoned attempts do not accumulate. The password is hashed before the pending registration is stored, so that even an unverified record does not hold a usable secret, and the code itself is stored only as a hash and compared in a way that does not reveal how much of a wrong guess was correct. Sign-in returns a short-lived access token and

a refresh token, and the refresh token is rotated on each use and stored only as a hash, so that a token captured in transit cannot be replayed after it has been used.

[TO BE PROVIDED: insert screenshot image for Figure 4.6]

Figure 4.6: Registration form

[TO BE PROVIDED: insert screenshot image for Figure 4.7]

Figure 4.7: Email verification code entry

4.2.5 Customer Account

A signed-in customer has a dashboard that lists their bookings, opens an individual booking, shows their profile and displays their notifications with an unread count. The account endpoints are scoped to the authenticated user, so that a customer can only see their own reservations, and a customer may update their own profile and cancel an eligible booking. This feature corresponds to the customer's need to manage a reservation after it has been made, and it is the point at which a guest who booked anonymously can associate the booking with an account.

The scoping is enforced on the server rather than by hiding operations in the interface. A request for a booking that belongs to another customer is answered as a missing record rather than being served, so that the existence of another guest's reservation is not disclosed. A customer may update a limited part of their own profile, namely the first name, the last name and the telephone number, while the email address is read-only and a request that tries to change it is refused. A customer may cancel a reservation only within the hotel's cancellation window, and the cancellation routes to the same rule that the front desk uses, so that the policy is applied identically however the cancellation is made.

[TO BE PROVIDED: insert screenshot image for Figure 4.8]

Figure 4.8: Customer dashboard listing bookings

[TO BE PROVIDED: insert screenshot image for Figure 4.9]

Figure 4.9: Customer booking detail and notifications

4.2.6 Profile and Notification Handling

Personal details are managed from the customer's own account and are restricted to the fields that the guest is entitled to change. The email address is deliberately read-only, because it is the address to which the verification code was sent and the address by which a booking is looked up, and allowing it to be changed without a new verification would weaken the identification of the account. A request that attempts to change it is refused with a structured error that the interface can present directly.

Notifications are created for a guest who holds an account, and they are raised when a reservation is confirmed, when a payment is recorded, and when a reservation is cancelled. A guest who booked anonymously does not receive an in-application notification, because there is no account to which to attach it, and such a guest receives the transactional email instead. The unread count is returned with the notification list so that the interface can show it without a second request, and reading a notification marks it as read through a dedicated endpoint.

4.2.7 Receptionist Console

The receptionist console is the operational heart of the system. It allows a member of staff to search and filter bookings, open a booking, create a walk-in booking, assign one or more physical rooms to each booked line, check a guest in and out, produce a check-out quote that includes any overstay charge, change a booked room type subject to availability, cancel a booking with a reason, and record a payment against the booking. Each of these operations moves the booking through its states in a controlled way, and room assignment is refused if a requested room conflicts with another reservation. This console addresses the research

question on supporting front-desk operations and is the part of the system that most directly replaces the manual processes analysed in Chapter Three.

The console is built around the reservation as a single object of work. The list can be filtered to the reservations that still owe money and searched by the guest's name or by the booking reference, so that the member of staff can find the reservation that a guest is standing at the desk to discuss. Opening a reservation shows its rooms, its payments and the actions that are legal in its current state, so that an operation that cannot apply is not offered. The state of the reservation is decided by the server, and the interface reflects that decision rather than predicting it, which keeps the console and the API in agreement about what is possible.

[TO BE PROVIDED: insert screenshot image for Figure 4.10]

Figure 4.10: Reception console booking list and search

[TO BE PROVIDED: insert screenshot image for Figure 4.11]

Figure 4.11: Room assignment and check-in

[TO BE PROVIDED: insert screenshot image for Figure 4.12]

Figure 4.12: Check-out with the overstay charge and payment recording

4.2.8 Walk-in Booking

A walk-in booking is created at the desk for a guest who arrives without a reservation, and it is distinguished from an online booking by its source. The receptionist enters the guest's details, chooses the room type and quantity, and, because the guest is already present, the check-in time is taken as the current time rather than being rejected as being in the past. The booking is then handled by the same assignment, check-in and check-out flow as an online booking, which keeps a single set of rules for both channels.

The walk-in flow is the one case in which a reservation is created without an email address, because the guest is present and will be served immediately, and it is for that reason that the confirmation is sent to the member of staff rather than to the guest. The guest record created

by a walk-in is kept separate from the records of guests who hold accounts, so that a walk-in never becomes attached to a registered customer by a coincidental match of an address. The reservation then follows the same path as any other: it may be assigned to rooms, checked in and checked out, and it consumes availability while it is held.

4.2.9 Check-in, Check-out and Overstay Charge

Check-in requires that the booking is confirmed and that at least one room has been assigned, after which the booking is marked as being in house and the assigned rooms are marked as occupied. Check-out produces a quote that re-prices the stay using the duration-based model described in Chapter Three, so that any time beyond the booked nights is charged according to the room type's tier table, and it records the final amount against the booking before releasing the assigned rooms and marking the booking as complete. Early check-out is supported and recorded with a reason, and both the actual check-in and check-out instants are stored so that the stay can be reconstructed.

The overstay charge is calculated from the actual check-out instant and the booked check-out instant, both of which are held in the booking, so that the amount does not depend on when the calculation is performed but only on the times that are recorded. A check-out that occurs before the booked moment is treated as an early check-out, which requires the consent of the guest and is stored with a reason, while a check-out after the booked moment is charged according to the tiers of the room type. When the hotel chooses to require payment before check-out, the operation is refused while a balance remains, so that a guest cannot leave with an unpaid balance when the hotel has asked for the gate to be enforced.

4.2.9.1 Room Assignment, Reassignment and Room-Type Change

The assignment of a physical room is a separate operation from the reservation, and it is performed at the desk shortly before the guest arrives. When a room is assigned, the server

checks that the room is of the required room type, that it is active and not under maintenance, and that it is free for the whole stay, and it refuses the assignment if another live reservation overlaps the range. A room may be reassigned while the guest is in house, in which case the old room is released and the new room is marked as occupied, and an unassigned room is released for future reservations.

A booked room type may be changed before check-in, subject to availability, and the change is recorded in the same way. The operation is refused if the new room type cannot accommodate the requested quantity for the stay, and it is refused if the amount already paid would exceed the new total, so that a change cannot leave a reservation in a state where the guest has paid more than the stay now costs. The change writes an audit entry and produces an updated confirmation for the guest, which keeps the record of the reservation consistent with what the guest has been told.

4.2.10 Payment Recording

A payment is recorded against a booking with its amount, method, status and any reference or note, and the staff member who recorded it and the time it was paid are stored with it. The supported methods are cash, point-of-sale, bank transfer and online, and the payment status of the booking is kept consistent with the payments recorded against it. Because the study did not integrate an online payment gateway, every payment is taken at the hotel and then recorded in the system, which keeps the payment record tied to the reservation it belongs to.

The outstanding balance of a reservation is computed on the server from its total amount and the payments that have been made against it, and it is returned with the reservation so that the console does not compute it. A payment cannot be recorded against a reservation that has been cancelled, because a cancelled reservation has no balance, and a payment cannot be recorded for more than the outstanding balance. These rules keep the sum of the payments

aligned with the amount of the reservation and prevent the two from drifting apart, which was the weakness of the manual record that the system replaces.

4.2.11 Administrator Console

The administrator console provides the management functions of the system. It allows an administrator to create, edit, activate and deactivate rooms and room types, to set the price, occupancy, features, images and overstay surcharge tiers of a room type, to list guests, payments and bookings, to restore a cancelled booking, to create and manage staff accounts and change a user's role, to edit the hotel's settings and stay policy, to view a dashboard of key figures and a reports view, and to review the activity log. These functions correspond to the administrator's use cases in Chapter Three and give the hotel the reports and the traceability that the manual system could not provide.

The room and room-type management is the part of the console that the hotel uses most, because it maintains the inventory against which every reservation is measured. A room type carries its nightly price, its capacity, its features and images and its overstay tiers, and a room carries its type and its operational state, so that a room can be withdrawn from sale without deleting the history of the reservations that used it. The staff management allows the appointment of a receptionist or an administrator and the change of a user's role, and every such change is written to the activity log together with the actor who made it, so that the grant of a privilege can be reviewed after the fact.

[TO BE PROVIDED: insert screenshot image for Figure 4.13]

Figure 4.13: Administrator dashboard and reports

[TO BE PROVIDED: insert screenshot image for Figure 4.14]

Figure 4.14: Room and room-type management, including surcharge tiers

[TO BE PROVIDED: insert screenshot image for Figure 4.15]

Figure 4.15: Staff management and activity log

4.2.12 Notifications and Email

Transactional messages are sent by email, and in-application notifications are stored for the user or guest to read. The email service sends through the configured mail server when one is available and falls back to writing the message to the application log when it is not, so that a missing mail configuration does not break a user journey such as registration. A background service expires stale pending registrations and adjusts booking statuses as time passes, so that abandoned verifications and overdue bookings do not accumulate.

The messages are built from a shared layout, so that a confirmation, a cancellation, a payment receipt and a check-out note share the same structure and the same visual identity rather than each being written separately. Each dynamic value in a message is encoded before it is included, so that a guest cannot inject content into a message through a name or a note, and no password is ever included in a message. The email service is defined by an interface, and the implementation that writes to the log is used when no mail account is configured, so that the system can be run without an external mail service while still exercising the same code path that a real send would take.

4.3 API Implementation

The frontend is built entirely on the versioned HTTP API, and there is no server-side rendering of the user interface. Requests and responses are JSON, and every error is returned as a Problem Details object with a machine-readable code, which allows the frontend to present a specific message and to distinguish a validation failure from a conflict (Nottingham & Wilde, 2016). Authentication uses a bearer token, and a refresh token is rotated and stored only as a hash. The endpoints are grouped by controller as shown in Table 4.1, and each declares its access requirement, so that the public catalogue and availability routes are

anonymous while the administration routes require the administrator policy. This arrangement makes the API the single contract between the two applications and keeps the business rules on the server.

The contract uses a small and consistent set of conventions. Property names are written in camel case, enumerations are serialised as their names, dates are written as a plain date and money as a decimal number, and every list is returned in a paged envelope that carries the items, the page number, the page size, the total count and the total number of pages. A list endpoint accepts a sort field and a direction and falls back to a sensible default when either is omitted, so that a client can request an ordering without knowing the full set of fields that may be sorted. These conventions are documented in the project's source of truth, which is the agreement that both the frontend and the backend are written against.

The API is versioned in its path, so that a future change to the contract can be introduced alongside the current one rather than replacing it. The identity of a caller is established by the access token that the frontend presents, and the role that the token carries is what the authorisation policies test. A token that has expired is replaced through the refresh endpoint without asking the guest to sign in again, and the refresh token is rotated on each use so that a token that has been used once cannot be used again. This combination gives the frontend a seamless session while keeping the lifetime of the access token short.

4.4 Functional Evaluation

The evaluation of the system is presented objective by objective, because those objectives are what the project set out to achieve. The evidence for each is the implemented behaviour observed through the interface or through an automated test, and the report states which.

The first objective, which concerned the management of the hotel's room inventory and room types, is met by the administration console's room and room-type management, which

supports creation, editing, activation and deactivation together with pricing, occupancy, features, images and surcharge tiers. The behaviour was observed through the interface and is covered by the administration integration tests.

The second objective, to provide a means for a guest to search availability and book online, is met by the availability search together with the booking flow, which returns available quantities and a price for a chosen stay, creates the reservation and issues a reference. The pricing logic is covered by unit tests and the booking flow by integration tests; the interface was exercised manually.

The third objective, to record reservations, guests and payments accurately and to prevent overbooking, is met by the relational model, the transactional booking creation with row-level locking and the payment records that are tied to a booking. Overbooking is prevented by the lock-and-recheck design described in Chapter Three; the idempotency and conflict behaviour is the subject of dedicated integration tests, which could not be executed in the environment used for this report, so this objective is supported here by the design and the unit-level evidence rather than by an executed integration run.

The fourth objective, to provide a front-desk console, is met by the reception module, which supports booking search, room assignment, check-in, check-out, overstay quoting, room-type change, cancellation and payment recording. These operations were exercised manually against the API and are covered by the reception integration tests.

The fifth objective, to provide an administrative interface, is met by the administration module described in Section 4.2.11, which covers rooms, room types, staff, settings, guests, payments, reports, dashboard and the activity log.

The sixth objective, to evaluate the system, is met by this chapter, which reports the automated tests that were executed, examines the reliability and security controls, and states the limits of the evaluation.

The evaluation distinguishes three kinds of evidence. The first is an executed automated test, which is the strongest evidence available because it is repeatable and it states its own passing or failing. The second is a manual observation of the running system, which is weaker because it was performed once by the developer and was not scripted. The third is a design argument supported by inspection of the code, which is the weakest but is still useful where an executed test was not possible. Each objective is placed against the strongest kind of evidence that was actually available, and no objective is described as verified on the strength of a test that was not run.

4.5 Reliability and Security Evaluation

Two reliability controls were built into the system and are the subject of dedicated tests and manual observation. The first is the prevention of overbooking through pessimistic row locks taken in a consistent order inside the booking transaction, which ensures that two concurrent requests cannot both succeed when only one room remains (PostgreSQL Global Development Group, n.d.). The second is idempotent booking creation, which makes it safe for a client to retry a request after a network failure; the same key returns the original booking, and the same key with a different body is rejected. Both controls are implemented in a single service so that the rule is defined once, and both are covered by the integration suite, which was not executed in this environment.

The two controls answer the two ways in which a booking system can produce a wrong record. A concurrent request can exceed the available capacity if each request reads the same remaining quantity before either writes, which is prevented by taking the lock before the read

that decides, and the lock is taken in a fixed order so that two transactions cannot each wait for the other. A retried request can create a second reservation for the same intent, which is prevented by making the request's identity part of the stored data and refusing to create a second booking for the same identity. Together the two controls address the two failure modes that the analysis of the existing system found to be the most damaging.

Three security controls were examined. Role-based authorisation is enforced on the server through named policies, so that a customer cannot reach staff or administrative operations and a receptionist cannot reach administrator-only operations; this was confirmed by inspecting the policy attached to each endpoint and by the responses returned to requests that lacked the required role. Credential handling was implemented with a hashed password, a refresh token stored only as a hash and rotated on use, and a keyed, constant-time-compared verification code, which follows the guidance of the OWASP Foundation (2021). Account enumeration is limited by returning an identical response for a forgotten-password request whether or not the address exists.

One weakness was identified in the working environment: development configuration files that were committed to the repository contained third-party credentials and a mail password, and although those values are not reproduced in this report, they should be treated as exposed and rotated, as recommended in the project's evidence register and future-work list. The finding is important because it shows that the security of a system does not rest on its code alone: a correct implementation of hashing and authorisation can still be undermined by a secret that is written into a file and placed under version control. The recommendation that follows is that the exposed values be rotated and that all future configuration be supplied through user secrets or environment variables, and it is carried into Chapter Five.

4.6 Testing Results

Table 4.2 records the automated tests present in the codebase and the outcome of the runs that were possible in the environment used for this report.

Suite	Scope	Tests	Result in this environment
Backend unit (xUnit)	Domain and application logic: paging, email templates, booking-reference generation, stay formatting and duration-based pricing	61 tests	Executed: 61 passed, 0 failed
Backend architecture (NetArchTest)	Layer dependency rules for Domain, Application and Infrastructure	3 tests	Executed: 3 passed, 0 failed
Backend integration (xUnit and Testcontainers)	API flows across availability, booking, idempotent duplicate booking, pricing quote, reception, walk-in, time-aware stays and administration, against a real database	130 test methods	Not executed: requires a running PostgreSQL instance or Docker, neither of which was available
Frontend unit (Vitest)	Utility and feature tests: dates, formatting, theme contrast and the verification countdown	95 tests across 4 files	Executed: 95 passed, 0 failed
Frontend static	Type checking (tsc --noEmit) and linting (eslint)	Not applicable	Executed: both completed with no errors
Frontend build	Production build (tsc -b and vite build)	Not applicable	Executed: build completed successfully

Table 4.2: Automated test suites and the outcome of the executed runs

The status of the principal scenarios is summarised in Table 4.3. Three statuses are used: "Passed", for a scenario exercised by an executed automated test; "Not executed", for a scenario whose test exists but could not be run in this environment; and "Not verified", for a scenario supported only by code inspection or an unscripted walkthrough.

ID	Feature or scenario	Status	Verification method
S1	Paging and list-envelope conventions	Passed	Automated unit test
S2	Transactional email templates, encoding and preheaders	Passed	Automated unit test
S3	Booking-reference format (DHI year and six characters)	Passed	Automated unit test
S4	Stay date and time formatting in the Lagos offset	Passed	Automated unit test
S5	Duration-based stay pricing and overstay tiers	Passed	Automated unit test
S6	Layer dependency rules	Passed	Automated architecture test
S7	Availability overlap rule over a real database	Not executed	Automated integration test (no database)
S8	Booking creation and idempotent duplicate handling	Not executed	Automated integration test (no database)
S9	Concurrent booking without overbooking	Not executed	Automated integration test (no database)
S10	Reception flow: assignment, check-in, check-out and payment	Not executed	Automated integration test (no database)
S11	Walk-in booking and time-aware stays	Not executed	Automated integration test (no database)
S12	Administration and role-policy enforcement	Not executed	Automated integration test (no database)
S13	Date, formatting and theme utilities in the client	Passed	Automated frontend test
S14	Type checking, linting and production build	Passed	Frontend tooling (tsc, eslint, vite)
S15	End-to-end journeys through the browser	Not verified	Developer walkthrough (not scripted)
S16	User acceptance testing with hotel staff	Not executed	No session conducted
S17	Load and response-time measurement	Not applicable	Out of scope for this study

Table 4.3: Verification status of the principal scenarios

The unit and architecture suites and the frontend checks were executed successfully, and the build completed without warnings being treated as errors. The integration suite could not be executed because it requires a PostgreSQL instance that was not available in the environment, and the report therefore makes no claim about its result. The functional journeys listed in Section 3.4.5 were exercised manually through the interface and behaved as described, but they were not recorded as a formal, scripted test pass, and no response-time or load measurements were taken. The consequence of these gaps is stated plainly: the reliability controls are supported by design and by unit-level evidence, and their confirmation under real concurrency remains to be demonstrated by an executed integration run.

The distinction between the suites matters for interpreting the result. The unit suite exercises the pure logic of the pricing and stay rules and does not need a database, so its success is a genuine result and not a partial one. The architecture suite exercises the dependency rules of the code and does not need a database either, so its success is likewise complete. The integration suite is the one that would demonstrate the concurrency and idempotency behaviour against a real database, and because it was not run, the design of those controls is supported by inspection and by the unit tests rather than by an executed end-to-end result. The report states this difference explicitly rather than presenting the whole system as tested.

4.7 Discussion: Alignment with Objectives and Research Questions

The system answers the research questions set in Chapter One. The question of a suitable design is answered by the layered, client-server architecture of Chapter Three, which separates the public site, the staff consoles and the business rules and enforces the separation with tests. The question of how availability should be determined so that a room cannot be booked twice is answered by computing availability on the server from the room inventory and the conflicting reservations, and by re-checking it under a row lock at the moment of

booking. The question of how reservations and guests should be recorded is answered by the relational model of Chapter Three, in which a booking relates a guest to one or more booking lines and, later, to assigned rooms.

The question of how front-desk operations should be supported is answered by the reception module, which presents the reservation as a single object of work and permits only the operations that are legal in its current state. The question of how the hotel's records should be managed is answered by the administration module, which maintains the inventory, the staff and the settings and produces the reports and the audit trail that the manual system could not. Taken together, the answers show that the objectives were met at the level of design and implementation, while the depth of the evaluation is limited by the unexecuted integration suite and the absence of user acceptance testing.

The alignment is not merely a matter of the features that were built. Each objective was answered by a specific design decision that can be pointed to: the room-inventory objective by the room-type and room management, the online-booking objective by the availability service and the booking flow, the overbooking objective by the transactional lock and the idempotency guard, the front-desk objective by the reception service, the administrative objective by the administration service, and the evaluation objective by this chapter. This mapping is what allows the evaluation to be read objective by objective rather than as a general claim of completeness.

4.8 Comparison with Related Systems

Compared with the related works, the system occupies the position that the gap analysis in Chapter Two identified. Like Williams and Micheal (2018) it uses a relational model and a layered architecture, but it adds an online guest channel with a live availability check and an explicit concurrency-control design. Like Mwandila (2026) it provides an availability check

and a staff-facing management interface, and it goes further by making the concurrency behaviour of booking explicit and testable. Unlike Booking.com (n.d.) and the commercial property-management products such as Cloudbeds (n.d.), it is a single-property application owned by the hotel rather than a marketplace or a subscription, and it implements the specific rules of the case study, such as the stay policy and the duration-based overstay pricing. Compared with the open-source QloApps (n.d.), it is smaller, written on a different stack, and aligned to the case study's workflow rather than to a general publishing model. Its distinctive contribution is the combination of a backend-authoritative availability and pricing engine with transactional, idempotent booking creation and a complete front-desk workflow in a single system for one small hotel.

The comparison can be made concrete along the three dimensions used in Chapter Two. On scope, the system is narrower than the commercial products and broader than the two academic systems, because it implements the whole front-desk workflow but for a single property. On availability and concurrency, it is more explicit than either academic system, because the locking and the idempotency guard are stated as design decisions and are covered by tests rather than left implicit. On ownership, it is neither a marketplace like Booking.com nor a subscription like Cloudbeds nor a general open-source platform like QloApps, but a system that the hotel owns and that implements the hotel's own rules.

4.9 User Acceptance Testing Status

No formal acceptance test with the hotel's staff had been carried out when this report was written. The functional journeys were exercised by the developer through the interface, and the outcomes were as described in Section 4.4, but no independent users evaluated the system and no satisfaction or usability data were collected. This report therefore describes the delivered behaviour and the automated evidence, and it does not claim a level of user

acceptance that was not measured. Conducting a structured acceptance session and recording its results is identified as the first item of future work in Chapter Five.

The absence of acceptance testing has a specific consequence for the reading of the evaluation. The functional evidence shows that the system behaves as designed, but it does not show that the design meets the expectations of the people who will use it, because the participants of a real acceptance session were not available at the time of writing. The report therefore separates the claim that the system works, which is supported by the evidence it presents, from the claim that the system is accepted, which is not yet supported, and it defers the second to the future work rather than merging the two.

4.10 Summary

This chapter presented the implemented system and evaluated it. It described the public booking flow, the customer account, the receptionist console and the administrator console, set out the API on which they are built, and evaluated the system against its objectives. It examined the reliability and security controls, reported the automated tests that were executed and those that could not be, aligned the results with the research questions, compared the system with the related works and stated the status of user acceptance testing. The evaluation shows that the system as designed and built meets its objectives and provides the controls the problem requires, while making explicit that the integration and acceptance evidence remains outstanding. Chapter Five draws the conclusions and sets out the recommendations and future work that follow.

CHAPTER FIVE

SUMMARY, CONCLUSION AND RECOMMENDATIONS

5.0 Introduction

This chapter concludes the report. It summarises what the study set out to do and what was produced, states the extent to which each objective was achieved, answers the research questions, and presents the findings that emerged from the work. It then offers recommendations to the case study and to others who may build on the system, identifies the work that remains, and closes with a brief statement on the value of the project. The chapter is written so that a reader who has followed the analysis, the design and the evaluation can see how the conclusions follow from the evidence rather than from a general claim of success.

5.1 Summary of the Study

The study set out to design and implement a web-based hotel management and booking system for Deutchezz Hotel Ispahan, a hotel in Jos, Plateau State, Nigeria, that was managing its rooms, reservations, guests and payments by hand. Chapter One established the problem, the aim, the objectives, the research questions and the scope and limitations of the study. Chapter Two reviewed the literature on hotel reservation systems, web architecture, databases and concurrency, security and agile development, and reviewed two academic reservation systems and three existing platforms, identifying the gap that this project addresses. Chapter Three analysed the manual system, stated the requirements, and presented the architecture, use cases, data model, modules, technology choices, implementation design and testing strategy. Chapter Four described the implemented features and the API, evaluated the system against its objectives, examined the reliability and security controls, and reported the tests that were executed. This chapter draws the work to a close.

The problem that motivated the study was the fragility of a manual record. When the availability of a room is held in a book and the same room type is requested by two people before either is written down, the hotel can promise the same room twice and discover the error only when the guests arrive. When prices and payments are recorded by hand, the amount a guest is shown and the amount recorded can diverge, and there is no reliable way to tell which is correct. The problem, in short, was not that the hotel lacked computers but that it had no single authoritative record against which every booking, price and payment could be measured.

The work was carried out in stages, following the wave order agreed at the start of the project, so that the foundation, the authentication, the booking domain and the interfaces were built in an order that allowed each to rest on the one before it. The design settled on a small and stable set of entities, a set of business rules stated in one place, and a backend that is the single authority for availability, price and state. The implementation produced two applications, a backend that exposes a versioned HTTP interface and a frontend that is built on it, together with an automated test suite that covers the domain logic, the architecture and the frontend. The evaluation examined the result against the objectives once more and stated plainly where the evidence is complete and where it is not.

5.2 Achievement of the Objectives

Each objective set in Chapter One was met in design and in implementation, with the qualification that the evaluation rests on the evidence available in the working environment. The room-inventory objective was met by the administration console's management of rooms and room types, including pricing, features, images and overstay tiers. The online-booking objective was met by the availability search and the booking flow that issues a reference and a confirmation. The recording-and-overbooking objective was met by the relational model,

by transactional booking creation with row-level locking, by idempotent creation and by payments that are tied to a reservation, while its confirmation under real concurrency remains dependent on an executed integration run. The front-desk objective was met by the reception console's assignment, check-in, check-out, room-type change, cancellation and payment features, and the administrative objective was met by the administration console's management, reporting and audit features. The evaluation objective was met by Chapter Four, which reports both the evidence and the limits of the evaluation.

The first objective, to design a system that manages the hotel's room inventory and room types, was achieved through a data model that separates the room type, which carries the price, the capacity, the features, the images and the overstay tiers, from the room, which carries the type and the operational state. This separation is what allows the hotel to change the price of a type without touching each room, and to withdraw a room from sale without deleting the history of the reservations that used it. The objective is demonstrated by the administration screens that create, edit, activate and deactivate rooms and room types and by the integration tests that exercise those operations.

The second objective, to provide a means for a guest to search availability and book online, was achieved through an availability service and a booking flow that share a single pricing engine, so that the price a guest is shown during the search is the price the booking will use. The booking flow issues a reference in a human-readable form, records the guest and the requested lines, and sends a confirmation, and it does not require the guest to hold an account beforehand, so that a first-time visitor can complete a reservation without an obstacle between the search and the confirmation.

The third objective, to record reservations, guests and payments accurately and to prevent overbooking, was achieved through the relational model, the transactional creation of a

booking under row-level locks taken in a consistent order, the idempotency guard that turns a repeated submission into a single reservation, and the payment records that are attached to the reservation they settle. The accuracy of the record follows from the fact that the total amount, the payments and the outstanding balance are computed by the server from the stored data, and the prevention of overbooking follows from the fact that availability is re-checked under the lock rather than trusted from the search that preceded the booking.

The fourth objective, to provide a front-desk console, was achieved through the reception service and the console built on it, which presents the reservation as a single object of work, offers only the operations that are legal in its current state, and records each action against the reservation. The console supports the operations that the manual process performed, namely finding a reservation, assigning a room, checking the guest in and out, quoting an overstay, changing a booked room type and recording a payment, and it adds the audit trail that the manual process lacked.

The fifth objective, to provide an administrative interface, was achieved through the administration module, which maintains the inventory, the room types, the staff accounts and the settings, and which produces the dashboard, the reports and the activity log. The sixth objective, to evaluate the system, was achieved by the preceding chapter, which reports the tests that were executed, states the tests that could not be executed and the reason, examines the reliability and security controls, and aligns the results with the objectives and the research questions.

5.3 Answers to the Research Questions

The question of a suitable design is answered by a layered, client-server architecture in which a typed React client speaks to a versioned ASP.NET Core API that holds the business rules and enforces its own layering with automated tests. The question of how available rooms

should be determined is answered by computing availability on the server from the room inventory and the conflicting reservations and by re-checking it under a row lock at the moment of booking, so that a physical room cannot be booked twice. The question of how reservations and guests should be recorded is answered by a relational model in which a booking links a guest to the room types requested and, later, to the physical rooms assigned, with payments attached to the booking. The question of how front-desk operations should be supported is answered by a single console that moves a booking through its states and records each action, and the question of how the inventory and administrative records should be managed is answered by an administration console with reports and an audit log.

Each answer has a design decision behind it that can be pointed to rather than inferred. The design question is answered by the four-layer split and by the architecture tests that fail if a layer is depended on from above. The availability question is answered by the half-open overlap rule together with the lock-and-recheck sequence, which are the two parts of the answer that make a double booking impossible rather than merely unlikely. The recording question is answered by the booking, booking-line, room-assignment and payment entities and by the rule that every amount is computed by the server. The front-desk question is answered by the state model of a booking, which decides which operation is legal at which point. The management question is answered by the settings provider, the reports and the activity log, which together give the hotel a way to change its rules and to see who changed what.

5.4 Findings

Several findings emerged from the work. First, keeping availability and price on the server, in one place, removed the need for the client to duplicate business rules and eliminated a class of discrepancy between what a guest is shown and what is charged. Second, the

prevention of overbooking is a concurrency problem rather than a data-entry problem, and it is solved reliably only by locking the affected rows in a consistent order inside the transaction that creates the booking, supported by an optimistic token on the paths that do not lock. Third, making booking creation idempotent is a small addition that removes a serious failure mode, namely the duplicate booking created when a client retries after a network timeout. Fourth, a duration-based overstay price can be expressed precisely with whole-minute boundaries and a validated, contiguous tier table, and snapshotting the applied tier keeps historical amounts stable. Fifth, the security controls that matter most for this kind of application are server-side authorisation and careful credential handling, and the practical weakness observed was not in the code but in committed development configuration, which must be rotated. Finally, a well-structured automated test suite is necessary but not sufficient: the reliability claims rest on integration tests that must actually be run against a real database.

The first finding has a consequence that goes beyond this system. When the client is allowed to compute a price or an availability, the same rule exists in two places, and the two copies drift apart as the system changes; the discrepancy that follows is then visible to the guest as a price that changes at the last step or an outcome that differs between two screens. Keeping one authority removes that whole class of problem, and it is the reason the frontend in this system renders what the backend reports rather than deciding for itself.

The second and third findings belong together, because they describe the two ways in which a duplicate or an inconsistent reservation can arise. A duplicate caused by concurrency is prevented by locking, and a duplicate caused by a retry is prevented by idempotency, and neither control substitutes for the other. The fourth finding is specific to the case study but generally useful, namely that a rule as apparently awkward as an overstay surcharge can be made exact if it is expressed in whole minutes and its tiers are validated to be contiguous, so that no stay can fall between two tiers.

The fifth finding is the one most easily overlooked. The code handled passwords, tokens and access correctly, yet a secret was found in a development file that had been placed under version control. The lesson is that the security of a system is a property of how it is operated as much as of how it is written, and that a single exposed secret can undo a great deal of careful implementation. The final finding is methodological: the existence of a test suite is not the same as the execution of a test suite, and a report that treats the two as equivalent misleads its reader. This report therefore states which tests ran and which did not.

5.5 Conclusion

The project demonstrated that a small hotel can be given a dependable, owned booking and management system using mainstream, freely available technologies, and that the important qualities of such a system, namely accurate availability, correct pricing, prevention of overbooking and controlled access, are engineering decisions that can be designed in from the start. The system centralises the hotel's records, gives guests a self-service booking channel, gives the front desk a reliable workflow and gives management reports and traceability. Its limitations are equally clear: the integration and acceptance evidence was not completed in the environment used for this report, no online payment was integrated, and the system is scoped to a single property. Within that scope, the objectives were achieved and the research questions answered.

It is worth stating what the project does not claim, because a report of this kind can easily overstate its result. It does not claim that the system has been proved correct under real concurrent load, because the integration suite that would demonstrate that was not executed; it does not claim that the system has been accepted by the hotel's staff, because no acceptance session was held; and it does not claim that the system is complete, because the manual process it replaces included functions such as online payment that were deliberately placed

outside the scope. What the project does claim is that the design addresses the problem it set out to solve, that the implementation realises that design, and that the evaluation shows where the evidence is strong and where it must be strengthened. That, rather than a claim of immaculate completion, is the honest result of the work.

5.6 Recommendations

It is recommended that the case study adopt the system in stages, beginning with the room and room-type data and the front-desk workflow, and that it run the integration test suite against a dedicated database before relying on the system in daily operation. It is recommended that the credentials found in committed development configuration be rotated immediately and that secrets be supplied through environment configuration in future. It is recommended that a structured user acceptance session be conducted with the front-desk staff and the management and that its results be recorded, so that the system can be refined against real use. It is also recommended that backups of the database be scheduled and tested, and that the staff be given brief training on the reception and administration consoles.

These recommendations are ordered deliberately. The rotation of the exposed credentials and the execution of the integration suite come before any reliance on the system, because both bear directly on whether the system can be trusted with real reservations. The staged adoption follows, so that the hotel does not place its whole operation on the system before the staff are familiar with it and before the inventory data has been entered and checked. The acceptance session then provides the evidence that the system meets the expectations of its users, and the training and the backups make the system sustainable once it is in daily use. A recommendation that is purely technical, such as the use of environment configuration, is paired with one that is operational, such as the schedule and test of backups, because the failure of a system is as likely to be organisational as it is technical.

For others who build on this work, one further recommendation is offered. The value of the design lies in the decisions rather than in the code, and a reimplementation should preserve the decisions even if it changes the technology. The decisions that matter are that the server is the single authority for availability, price and state; that booking creation is transactional, ordered and idempotent; that every amount is computed rather than accepted from the client; and that access is enforced on the server rather than in the interface. A system that keeps these decisions will keep the qualities that this project sought, whatever framework it is written in.

5.7 Suggestions for Further Work

Several extensions would strengthen the system. The first is the completion and continuous execution of the integration test suite against a real database, since this is the evidence on which the reliability claims depend. The second is a structured user acceptance evaluation with the case study. The third is the integration of an online payment gateway so that a guest can pay at the point of booking, with the payment reconciled automatically against the reservation. The fourth is the addition of housekeeping and room-maintenance workflows, so that a room's operational status is managed alongside its reservations. The fifth is a management reporting and analytics layer that produces the occupancy, revenue and cancellation figures that the case study currently assembles by hand. Beyond these, support for multiple properties and for dynamic or seasonal pricing would extend the system to larger operations, and the concurrency controls could be exercised under a load test to demonstrate their behaviour at realistic volumes.

The first three of these extensions are the ones that would most change the strength of the project's claims. Completing the integration suite would turn the reliability claims from arguments into results, an acceptance evaluation would replace the absence of user evidence

with measured findings, and an online payment gateway would close the one gap between the implemented flow and the full booking journey that a modern guest expects. The remaining extensions, the housekeeping and analytics functions and the larger-scale features, would broaden the system rather than deepen its foundation, and they are therefore placed after the first three.

There is also a place for work that is more exploratory. A study could compare the lock-based approach taken here with an optimistic alternative under a realistic mix of bookings, to see where each is preferable, and it could measure the effect of the idempotency guard on the rate of duplicate reservations in a live setting. A further study might examine how a system of this kind is adopted by a small hotel over time, since the organisational factors that determine whether such a system is used well are as important as the engineering decisions that make it work.

5.8 Closing Remarks

This report has shown how a manual hotel operation can be replaced by a web-based system that keeps a single, accurate record of rooms, reservations and guests, that prevents the overbooking that the manual method could not, and that serves both the guest and the hotel's staff. The design decisions that make this possible are not exotic; they are the careful application of ordinary and well-established engineering practice to a domain whose central risk is the consistency of its data. The system is a working foundation, and the work that remains is well defined and within reach.

The study leaves behind not only a working system but a clear account of why it was built the way it was, so that the next person to work on it has both the artifact and the reasoning behind it. That pairing of a system with a defensible account of its design is, in the end, what makes the work useful beyond the case study for which it was written.

REFERENCES

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). Manifesto for agile software development. Retrieved from https://agilemanifesto.org/

Booking.com. (n.d.). About Booking.com. Retrieved from https://www.booking.com/

Cloudbeds. (n.d.). Cloudbeds property management system. Retrieved from https://www.cloudbeds.com/property-management-system/

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures (Doctoral dissertation, University of California, Irvine).

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Reading, MA: Addison-Wesley.

Kleppmann, M. (2017). Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems. Sebastopol, CA: O'Reilly Media.

Microsoft. (2023). Handling concurrency conflicts - EF Core. Retrieved from https://learn.microsoft.com/en-us/ef/core/saving/concurrency

Mwandila, E. (2026). Design and development of an online hotel reservation system: Case study of Lundazi Castle Hotel and its branches. International Journal of Advanced Multidisciplinary Research and Studies, 6(2). doi:10.62225/2583049X.2026.6.2.6059

Nottingham, M., & Wilde, E. (2016). Problem details for HTTP APIs (RFC 7807). Internet Engineering Task Force. doi:10.17487/RFC7807

OWASP Foundation. (2021). OWASP Top 10:2021. Retrieved from https://owasp.org/Top10/2021/

PostgreSQL Global Development Group. (n.d.). PostgreSQL 18 documentation: Explicit locking. Retrieved from https://www.postgresql.org/docs/18/explicit-locking.html

QloApps. (n.d.). QloApps - Open-source hotel management system [Computer software]. Retrieved from https://github.com/Qloapps/QloApps

Sommerville, I. (2016). Software engineering (10th ed.). Boston, MA: Pearson.

Williams, K., & Micheal, A. (2018). Design and implementation of reservation management system: Case study: Grand Ville Hotels. Journal of Information Technology & Software Engineering, 8(4). doi:10.4172/2165-7866.1000243

APPENDIX A: API ENDPOINT CATALOGUE

This appendix lists every endpoint action exposed by the backend, grouped by the controller that owns it. The counts agree with Table 4.1 of Chapter Four, namely ten controllers and sixty-six endpoint actions. The access shown is the requirement declared by the endpoint: Public endpoints are anonymous, Authenticated endpoints require any signed-in caller, Customer endpoints are scoped to the authenticated customer, Staff endpoints admit a receptionist or an administrator, and Administrator endpoints admit only an administrator. All routes are relative to the versioned base path api/v1.

Controller	Method	Route	Access
Auth	POST	/auth/register	Public
Auth	POST	/auth/verify-email	Public
Auth	POST	/auth/resend-code	Public
Auth	POST	/auth/login	Public
Auth	POST	/auth/refresh	Public
Auth	POST	/auth/forgot-password	Public
Auth	POST	/auth/reset-password	Public
Auth	POST	/auth/logout	Authenticated
Auth	GET	/auth/me	Authenticated
Me	GET	/me/bookings	Customer
Me	GET	/me/bookings/{id}	Customer
Me	POST	/me/bookings/{id}/cancel	Customer
Me	GET	/me/profile	Authenticated
Me	PUT	/me/profile	Authenticated
Me	GET	/me/notifications	Customer
Me	GET	/me/notifications/unread-count	Customer
Me	POST	/me/notifications/{id}/read	Customer
Bookings	POST	/bookings	Public or Authenticated
Bookings	POST	/bookings/lookup	Public
Bookings	POST	/bookings/cancel	Public
Availability	POST	/availability/search	Public
Pricing	POST	/pricing/quote	Public
Room types	GET	/room-types	Public
Room types	GET	/room-types/{id}	Public
Stay policy	GET	/stay-policy	Public
Reception	GET	/reception/bookings	Staff
Reception	GET	/reception/bookings/{id}	Staff
Reception	POST	/reception/bookings	Staff

Reception	POST	/reception/bookings/{id}/cancel	Staff
Reception	POST	/reception/bookings/{id}/assign-rooms	Staff
Reception	POST	/reception/bookings/{id}/check-in	Staff
Reception	POST	/reception/bookings/{id}/check-out	Staff
Reception	POST	/reception/bookings/{id}/check-out-quote	Staff
Reception	POST	/reception/bookings/{id}/change-room-type	Staff
Reception	POST	/reception/bookings/{id}/reassign-room	Staff
Reception	POST	/reception/bookings/{id}/unassign-room	Staff
Reception	GET	/reception/rooms	Staff
Reception	GET	/reception/guests	Staff
Reception	GET	/reception/payments	Staff
Reception	POST	/reception/payments	Staff
Admin	GET	/admin/rooms	Administrator
Admin	GET	/admin/rooms/{id}	Administrator
Admin	POST	/admin/rooms	Administrator
Admin	PUT	/admin/rooms/{id}	Administrator
Admin	PATCH	/admin/rooms/{id}/status	Administrator
Admin	GET	/admin/room-types	Administrator
Admin	GET	/admin/room-types/{id}	Administrator
Admin	POST	/admin/room-types	Administrator
Admin	PUT	/admin/room-types/{id}	Administrator
Admin	PATCH	/admin/room-types/{id}/status	Administrator
Admin	GET	/admin/bookings	Administrator
Admin	POST	/admin/bookings/{id}/restore	Administrator
Admin	GET	/admin/guests	Administrator
Admin	GET	/admin/users	Administrator
Admin	PATCH	/admin/users/{id}/role	Administrator
Admin	GET	/admin/staff	Administrator
Admin	POST	/admin/staff	Administrator
Admin	PATCH	/admin/staff/{id}/status	Administrator
Admin	GET	/admin/payments	Administrator
Admin	GET	/admin/notifications	Administrator
Admin	GET	/admin/settings	Administrator
Admin	PUT	/admin/settings	Administrator
Admin	GET	/admin/dashboard	Administrator
Admin	GET	/admin/reports	Administrator
Admin	GET	/admin/activity-logs	Administrator
Uploads	POST	/uploads/images	Administrator

Table A.1: Complete API endpoint catalogue

APPENDIX B: CORE BUSINESS RULES REGISTER

This appendix restates the core business rules that the source of truth defines for the system. Each rule is enforced by the backend rather than by the interface, and each is the authority behind one or more of the design decisions and evaluations described in Chapters Three and Four.

Rule	Statement
BR-001	Anonymous booking is allowed without account creation.
BR-002	Registered customers use the same booking domain and can view their reservations in a dashboard.
BR-003	Registration may be encouraged but must not be mandatory for guest checkout.
BR-004	Anonymous booking lookup requires the booking reference together with the guest email address.
BR-005	A booking can contain multiple room types and quantities; there is no one-room-per-booking assumption.
BR-006	Customers reserve room types and quantities; physical rooms are assigned separately.
BR-007	A room is unavailable for a stay when a conflicting active reservation or assignment occupies the requested range.
BR-008	The check-in date is inclusive and the check-out date is exclusive for night and overlap calculations.
BR-009	Check-out must be after check-in; the number of nights is calculated on the server.
BR-010	The backend calculates prices and totals; client-submitted totals are never authoritative.
BR-011	Each booking line stores the agreed nightly rate as a snapshot at the time of booking.
BR-012	The booking status and the payment status are separate.
BR-013	A booking may have multiple payment records, and the paid amount is calculated from the valid records.
BR-014	Payment is recorded manually as cash, point-of-sale or bank transfer; an online gateway is deferred.
BR-015	Check-in marks the assigned rooms occupied and check-out releases them unless another operational status applies.

BR-016	Check-in validates the booking state, the dates, the assignments and the configured payment policy.
BR-017	Check-out validates that the booking is checked in and follows the configured outstanding-balance policy.
BR-018	Cancelled bookings do not consume room availability.
BR-019	Customer, receptionist and administrator permissions are enforced by the backend.
BR-020	Important administrative and operational actions generate audit records.
BR-021	The initial twenty rooms are sample inventory only; an administrator can add more.
BR-022	Booking and inventory changes use transactions and concurrency controls sufficient to prevent overbooking.
BR-023	A physical room cannot be assigned to two overlapping live bookings; a conflict is rejected with a structured conflict.
BR-026	An account is created only after the email-verification code is confirmed; unverified sign-ups are purged.
BR-027	Telephone numbers are stored normalised and are unique across accounts; a duplicate is rejected without revealing the holder.
BR-028	Registration, resend and forgotten-password responses never reveal whether an address is already registered.
BR-029	Authentication and code failures return a single generic message and do not disclose account state.
BR-030	A user holds exactly one role, and an administrator can change it through the role endpoint.
BR-031	Changing a role revokes existing refresh tokens so that the new role applies at the next sign-in.
BR-032	An administrator cannot change their own role.
BR-033	The system keeps at least one active administrator and rejects demoting or deactivating the last one.
BR-034	A receptionist or administrator can cancel a pending or confirmed booking with a reason, not bound by the guest window.
BR-035	A walk-in guest is stored with a name and telephone only, and is never linked to an account.
BR-036	A guest with no email receives no email; a walk-in summary is sent to the creating staff member instead.
BR-037	Payment list rows show the guest name derived

	from the booking, for online and walk-in bookings alike.
BR-038	A check-out before the scheduled moment is an early check-out and requires the guest consent flag and a reason.
BR-039	Before check-in a booking line can be moved to a different room type transactionally, re-pricing and re-checking availability.
BR-040	Before check-in a room may be reassigned or unassigned, and a reassignment also works while the guest is in house.
BR-041	A public booking that repeats the same guest within the duplicate window is rejected unless it is acknowledged.
BR-042	Personal details can be changed only by the account owner, and the email address is read-only.

Table B.1: Core business rules register